



DEPARTMENT OF ENGINEERING MATHEMATICS

Robustness of Bayesian Neural Networks Design Choices Against Weight and Bias Perturbation Induced by Single Event Upsets (SEUs)

Revalda Putawara

A dissertation submitted to the University of Bristol in accordance with the requirements of the degree
of Master of Science in the Faculty of Engineering.

Wednesday 17th September, 2025

Supervisor: Dr. James Pope

Declaration

This dissertation is submitted to the University of Bristol in accordance with the requirements of the degree of MSc in the Faculty of Engineering. It has not been submitted for any other degree or diploma of any examining body. Except where specifically acknowledged, it is all the work of the Author.

Revalda Putawara, Wednesday 17th September, 2025

Contents

1	Introduction	1
1.1	Background	1
1.2	Motivations	1
1.3	Our Approach	2
2	Background	3
2.1	Convolutional Neural Networks (CNN)	3
2.2	Bayesian Neural Networks	3
2.3	Robust ML Systems	6
2.4	Single Event Effects	7
2.5	Robustness of Neural Networks	8
2.6	Measure of Robustness	8
3	Design	9
3.1	Dataset Description	9
3.2	Deterministic CNN Model Architecture	9
3.3	Bayesian Implementation of the Deterministic CNN Model	10
3.4	Training Process	10
3.5	Inference Process	10
3.6	Activation Function Variations	11
3.7	Prior Distribution Parameter Variations	11
3.8	Model Variations	12
3.9	SEU process	12
3.10	Robustness Measure	14
4	Implementation	15
4.1	Experimental Environment	15
4.2	Computation Resource	15
4.3	Dataset Preparation	15
4.4	Training of Deterministic CNN Model	16
4.5	Training of Bayesian CNN Model	16
4.6	Implementations in Pyro	16
5	Critical Evaluation	17
5.1	Analysis of Model Performance Metric	17
5.2	Analysis of Performance Against SEU	19
6	Conclusion	27
6.1	Project Status	27
6.2	Limitations	28
6.3	Future Work	29
A	Appendix	35

List of Figures

2.1	Illustration of max and average pooling (2 x 2 filters and stride 2) as shown in [52]	3
2.2	Prior Probability Distribution Function for $a = 0$ and $b = 1$ (Gaussian and Laplace) and $a = -0.5$ and $b=0.5$ (Uniform)	5
2.3	Illustration of Bayesian Neural Networks, as shown in [9]	6
2.4	Ionization path due to direct passage of heavy iron (a), and due to proton reaction in a device (b) as shown in [42]	7
3.1	Sample images from the Shpnsnet dataset.	9
3.2	Activation Functions Plot	11
5.1	Robustness of Deterministic and Bayesian CNN model	19
5.2	Illustration of SEU injection and its propagated effect on conv1 layer (a) and fc1 layer (b) in a CNN model	25
A.1	Architecture for storing floating point numbers in this research, which is FP32	35
A.2	Sampled Weight Distribution before SEU (panel a), Uniform(-0.28,0.64) and after SEU (panel b), Uniform(1000.0,0.64)	35
A.3	Robustness of Deterministic CNN model by model variant	39
A.4	Robustness of Bayesian CNN model by model variant	40
A.5	Log mean absolute difference against robustness metrics, excluding the data from bit index=1	40

List of Tables

2.1	Prior distribution parameter used in this research	5
3.1	Layers in our model	10
5.1	Test accuracy of different activation functions across model variants in evaluating Deterministic CNN Models	17
5.2	Average test accuracy (mean across models) by prior and activation function. Best per prior b in bold.	18
5.3	Average test accuracy by model for prior $b = 1$. Best per model variant in bold.	18
5.4	Summary of average absolute accuracy difference and average softmax difference Deterministic vs Bayesian CNN	20
5.5	Robustness of Deterministic and Bayesian CNN model by model variant	21
5.6	Robustness of Bayesian CNN model by prior and prior parameter	22
5.7	Robustness of Deterministic and Bayesian CNN model by activation function	23
5.8	Robustness of Deterministic and Bayesian CNN model by layer and layer parameter	24
5.9	Robustness of Deterministic and Bayesian CNN model by attacked bit location	26
5.10	Robustness of Deterministic and Bayesian CNN model by original bit attacked	26
A.1	Train accuracy of different activation functions across model variants in evaluating Bayesian CNN Models $b=0.1$	36
A.2	Train accuracy of different activation functions across model variants in evaluating Bayesian CNN Models $b=1.0$	36
A.3	Train accuracy of different activation functions across model variants in evaluating Bayesian CNN Models $b=10.0$	37
A.4	Test accuracy of different activation functions across model variants in evaluating Bayesian CNN Models $b=0.1$	38
A.5	Test accuracy of different activation functions across model variants in evaluating Bayesian CNN Models $b=1$	38
A.6	Test accuracy of different activation functions across model variants in evaluating Bayesian CNN Models $b=10$	39
A.7	Illustration of value change after SEU (original value = -0.60335773229599)	40

List of Algorithms

3.1	Monte Carlo Sampling for Bayesian CNN Inference	11
3.2	SmartPool: Reliable Max-Pooling with Outlier Detection and Correction	13

Abstract

Bayesian Neural Networks (BNNs) introduce uncertainty into deep learning models by treating weights and biases as probability distributions instead of exact values, making them promising for deployment in harsh environments where reliability under hardware faults is crucial. Single Event Upsets (SEUs), caused by radiation, are a major fault source in machine learning systems. While prior work has examined deterministic CNNs under SEUs and BNNs under adversarial attacks, little research addresses BNN robustness against SEUs.

In this research, deterministic and Bayesian CNN models were trained under 4 different design choices: base model, modified pooling, dropout, and weight decay. Each design was tested with seven activation functions. For BNNs, three priors (Gaussian, Laplace, Uniform) were evaluated. Models were subjected to SEU injections targetting different layers, layer parameters, bit positions, and neuron positions to assess robustness. A new metric, the Aggregate Robustness Index (ARIn), was introduced to jointly measure accuracy difference and softmax difference.

Results show that deterministic CNNs achieve a higher baseline test accuracy (97.55% vs. 82.85%), but Bayesian CNN models showed superior robustness with an ARIn score of 0.0779 compared to Deterministic CNNs' 0.1909. Dropout improved BNN robustness the most, with Gaussian priors outperforming Laplace and Uniform. SEU attacks on the variance parameters were more damaging than those on means. Among activation functions, Sigmoid proved to be the most robust overall, though with an accuracy-robustness trade-off. Fully connected layers were more vulnerable, as SEU directly impacted the logits. First exponent bit-flips are the most harmful SEU, and flipping 1 to 0 showed a better robustness.

Overall, this work concludes that Bayesian CNNs are not universally more robust than deterministic CNNs, but they provide greater stability in prediction confidence. This makes them suitable for applications where class probabilities are crucial, such as medical imaging and ensemble learning systems.

Supporting Technologies

This work relied on a range of supporting technologies/third-party resources. The main tools are summarized below:

- Computation was carried on Amazon Web Services with Elastic Compute Cloud (EC2) to provision virtual machines with GPU.
- *Python* is used as the main programming language, with *Anaconda* to provide packages and virtual environment.
- Deep learning models were implemented with *PyTorch* (for deterministic CNN models) and *Pyro* (for Bayesian CNN models).
- Experiment result data analysis and visualization were supported by publicly available Python libraries such as *Pandas*, *Matplotlib*, and *Seaborn*.
- Version control was maintained through *Github*.
- This dissertation was formatted using L^AT_EX using the cloud-based editor *Overleaf*.

Notation and Acronyms

For clarity and ease of reference, this chapter summarizes the mathematical notation and acronyms used throughout the dissertation. The notation and acronyms are defined as follows and will be used consistently throughout this dissertation, unless explicitly stated otherwise.

BNN	:	Bayesian Neural Networks
CNN	:	Convolutional Neural Networks
DNN	:	Deterministic Neural Networks
SEE	:	Single Event Effect
SEU	:	Single Event Upset
WG	:	Weighted Gaussian
RWG	:	Rectified Weighted Gaussian
ELBO	:	Evidence Lower Bound
VI	:	Variational Inference
SVI	:	Stochastic Variational Inference
MCMC	:	Markov Chain Monte Carlo
HMC	:	Hamiltonian Monte Carlo
AAAD	:	Average Absolute Accuracy Difference
ASD	:	Average Softmax Difference
ARIn	:	Aggregate Robustness Index
$p(\theta)$	:	posterior distribution over parameters
$p(\theta y)$	:	posterior distribution over parameters given data
$q_\phi(z)$	:	variational distribution
$p_\theta(z x)$	:	true posterior
$\text{KL}(q_\phi(z) \ p_\theta(z x))$	:	Kullback-Leibler divergence between variational distribution and true posterior

Acknowledgements

First and foremost, I would like to express my deepest gratitude to my wife, Alleya, whose unconditional love and unwavering support have accompanied me every day throughout this journey. Her encouragement to pursue this master's program was the spark that set everything in motion, and her presence has been my anchor, helping me persevere until the very end.

I am profoundly grateful to the Indonesia Endowment Fund for Education Agency (LPDP) for placing their trust in me through the scholarship program. This opportunity has truly been life-changing and has opened doors I could only have dreamed of before.

My heartfelt thanks also go to Warga Bristol 2024, a community of fellow Indonesian students who began this journey alongside me. Their companionship, shared laughter, coffee sessions, comforting food that reminded me of home, and countless conversations about everyday matters have made this experience richer and more meaningful.

Finally, I extend my sincere appreciation to my supervisor, Dr. James Pope, whose guidance and constructive advice have continually motivated me to deliver my best work. His support has been invaluable in shaping the progress and outcome of this project.

Chapter 1

Introduction

This chapter introduces the foundations of the research, including the research background, motivation, objectives, and challenges. It begins with a discussion on machine learning models' role in real-world deployment under normal and harsh environments. The motivation of this research will then be stated, especially highlighting the limited existing work in examining the effect of weight/bias perturbation against Bayesian Neural Networks. Finally, we will present the approach of the research, by clearly stating the aim of the research and also the possible challenges that will be faced during the research.

1.1 Background

Machine learning models are widely used in our daily life. Most of machine learning systems can be found close to us, like a recommender system, large language model or even autonomous vehicle that utilizes Convolutional Neural Network (CNN) models to steer clear from obstacles. While most of those systems are deployed in a normal environment, some of machine learning models are also deployed in harsh environments with its own specific designated purpose.

One of machine learning system that is deployed in a harsh environment is CloudScout [23]. CloudScout, which is installed in Hyperscout-2 satellite, part of the European Space Agency (ESA) PhiSat-1 mission, is a first in-orbit demonstration of Deep Neural Network for data processing on edge. It uses CNN to classify hyperspectral satellite images as either cloudy or not cloudy directly on-board, and only transmit the non cloudy image to the ground, significantly reducing bandwidth consumption.

Machine learning systems deployed in harsh environments are exposed to risks during its inference, and a common term to associate this performance with Deep Neural Networks is *robustness* [48]. Robustness are categorized into two subproperties, security and reliability. While the security aspect of DNNs focuses on preventing information inside the network to be stolen or protecting the system to get an incorrect input (adversarial attack), the reliability aspect of robustness focuses on keeping the model to display a consistent output, parameter, or behaviour, despite changes of environmental condition or fabrication.

1.2 Motivations

Being deployed in the space, a harsh environment, we can assume that machine learning systems like CloudScout is prone to risks that exists due to being deployed outside of earth surface. One of the risks that could hamper the performance of machine learning models that are deployed in harsh environments is single event effects (SEEs).

One example of SEE that can happen to machine learning models deployed in a harsh environment is Single Event Upsets (SEUs). SEUs, explained in [42], are a temporary change of memory or control bit. One phenomenon that occurs and has shown the characteristic of SEUs is the incident of bitflips.

Machine learning with a good quality of robustness, or Robust Machine Learning Systems, can mitigate the impact of SEUs/bitflips, rendering the output of the model to be consistent. In this research, we are motivated to understand the impact of SEUs/bitflips to weight/bias perturbation in Bayesian Neural Networks.

This research is built upon work done in [15] that reviewed CNN design choices to develop robust machine learning models. This research is also motivated by previous research that has proved that Bayesian Neural Networks are more robust against adversarial attacks (security aspect of robustness).

Although there are already a significant number of studies that addressed those two aforementioned fields, **there has been little to no investigation into the robustness of Bayesian Neural Networks against weight/bias perturbation** (reliability aspect of robustness). This dissertation seeks to address that gap.

1.3 Our Approach

In this research, we will show multiple Bayesian Neural Network design choice and compare its performance measure. Subsequently, we will simulate Single Event Upset (SEUs) with bitflips across all Bayesian Neural Network design alternatives and measure its performance measures afterwards. Robustness is then measured from the difference on the result of the model, before and after the model is perturbed with SEUs.

1.3.1 Research Aim

To find a Bayesian Neural Network design choice that is robust against perturbation of weight/bias, we aim to complete several tasks in this research:

1. Train multiple deterministic and Bayesian Neural Networks with different design choices and note their performance
2. Do multiple experiment of SEU injection on the different deterministic and Bayesian Neural Network design and note its performance
3. Compare the robustness of each different design
4. Validate whether Bayesian Neural Networks are more robust compared to deterministic Neural Networks

1.3.2 Challenges

The challenge that is imminent in this research is mainly due to the Bayesian nature of the BNNs. The challenge also come from simulating the process of SEU. The challenges are as follows:

1. Due to the uncertainty of model weight and bias selection in Bayesian Neural Networks, the performance observed from it does not replicate its deterministic counterpart completely.
2. All models are designed to have all their layers and neurons to be approximated with the same parameter distribution, potentially making the resulting model sub-optimal towards the solution.
3. Magnitude of the result of SEU/bitflips might differ between neurons, making the measurement of robustness not a straightforward process.

Chapter 2

Background

2.1 Convolutional Neural Networks (CNN)

In machine learning models with images as its input, we can simply treat each pixel as channels of color intensity, and then build a fully connected network. In principle, this network could work and yield a good result, but ignore the key property of images, that nearby pixels are more correlated than pixels far away from it. Modern computer vision methods utilize this characteristic by extracting local features that depend only on small subregions of image. Convolutional neural network take advantage of this idea in three mechanisms: local receptive fields, weight sharing, and subsampling [7].

Convolutional layers use local receptive fields, with each unit looking only at a small patch (for example, 4 x 4) of the input image. All of the units in that layer share the same weights, with the purpose of detecting the same pattern at different locations. The output of this process is a feature map with kernels of the image, which became the input for the sub-sampling layer.

In the sub-sampling layer, the spatial resolution of feature maps is further reduced by pooling algorithms. There are some popular pooling algorithms that are used in the sub-sampling layer [52]:

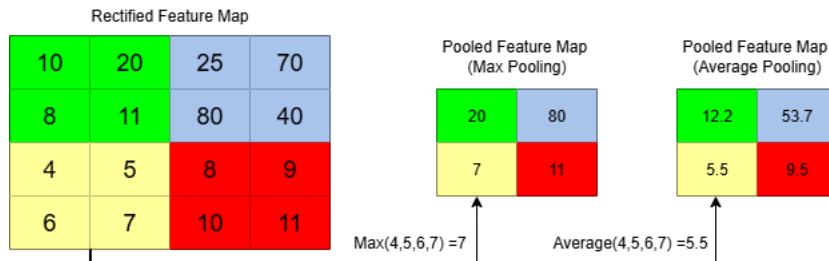
1. Max Pooling Method: Identifies the biggest element in each pooling region. The biggest limitation of max pooling lies in how all other items are ignored completely for the sake of the biggest value observed [41].

$$f_{\max}(X) = \max \{x_1, \dots, x_N\} \quad (2.1)$$

2. Average Pooling Method: Calculates the average values for each pooling region.

$$f_{\text{ave}}(X) = \frac{1}{N} \sum_{i=1}^N x_i \quad (2.2)$$

Figure 2.1: Illustration of max and average pooling (2 x 2 filters and stride 2) as shown in [52]



2.2 Bayesian Neural Networks

2.2.1 Bayes' rule and Bayesian Inference

Joint probability distribution for parameter θ and observed value y can be written as a product of two densities that are often referred to as the prior distribution $p(\theta)$ and the sampling distribution (or data distribution) $p(y|\theta)$ [22]: $p(\theta, y) = p(\theta) p(y | \theta)$.

Conditioning on the known value of the data y using the basic property of Bayes' rule, posterior density, can be notated in Equation (2.3). An equivalent form of Equation (2.3), in which omits the factor of $p(y)$, which can be considered a constant due to its independency from θ , yield the *unnormalized posterior density* of Equation (2.4).

While Bayes' rule is often used to infer unknown model parameters θ , Bayesian Inference is more general and can be applied to latent variables, evidence evaluations, and predictions. In this broader context, we often introduce latent variables z and compute distributions over both the observed data x and hidden variables z . θ defines the model, inference focuses on latent variables z , and θ is learned via evidence maximization. Drawing conclusions about latent variables from data can be done by computing posterior over latent variables z , as shown in Equation (2.5)

How well a model fits observed data x can be quantified with Evidence (marginal likelihood), which can be defined with Equation (2.6). To make predictions for new data, we can utilize the posterior predictive distribution in Equation (2.7). In order to learn the parameters θ of our models from observed data x , we can maximize the marginal likelihood as shown in Equation (2.8).

$$p(\theta | y) = \frac{p(\theta, y)}{p(y)} = \frac{p(\theta) p(y | \theta)}{p(y)} \quad (2.3) \quad p(\theta | y) \propto p(\theta) p(y | \theta) \quad (2.4)$$

$$p_\theta(z | x) = \frac{p_\theta(x, z)}{\int dz p_\theta(x, z)} \quad (2.5) \quad p_\theta(x) = \int dz p_\theta(x, z) \quad (2.6)$$

$$p_\theta(x' | x) = \int dz p_\theta(x' | z) p_\theta(z | x) \quad (2.7) \quad \theta_{\max} = \arg \max_{\theta} p_\theta(x) = \arg \max_{\theta} \int dz p_\theta(x, z) \quad (2.8)$$

2.2.2 Prior Distribution

Prior ($p(\theta)$) represents the belief about parameter θ before observing any data. If this prior shows genuine, context-specific information about the parameter, it is referred as an **informative prior**. The choice of prior not only shows our assumptions or existing knowledge but can also influence the posterior result. To see the effect on choosing a prior to the posterior, we observe three types of prior distributions.

Gaussian distribution/Normal distribution is highly mentioned with the central limit theorem, which states that if a random variable is the sum of a large number of independent random variables, it is normally distributed [46]. The density function of parameter θ that depends of two parameters, mean μ and standard deviation, σ (where $-\infty < \mu < \infty$, $\sigma > 0$) is shown in Equation (2.9).

Laplace distribution is a heavy-tailed distribution that is also known as double-sided exponential distribution [38]. The density function of the Laplace distribution is given by Equation (2.10). μ is a location parameter and $b > 0$ is a parameter of scale. Laplace distribution is deemed to be more robust to outliers compared to the Gaussian distribution [38].

If we know nothing about θ , we can use a **Uniform** prior, which is a kind of uninformative prior [38], as shown in Equation (2.11). The uniform distribution can be represented as a beta distribution with $a = b = 1$.

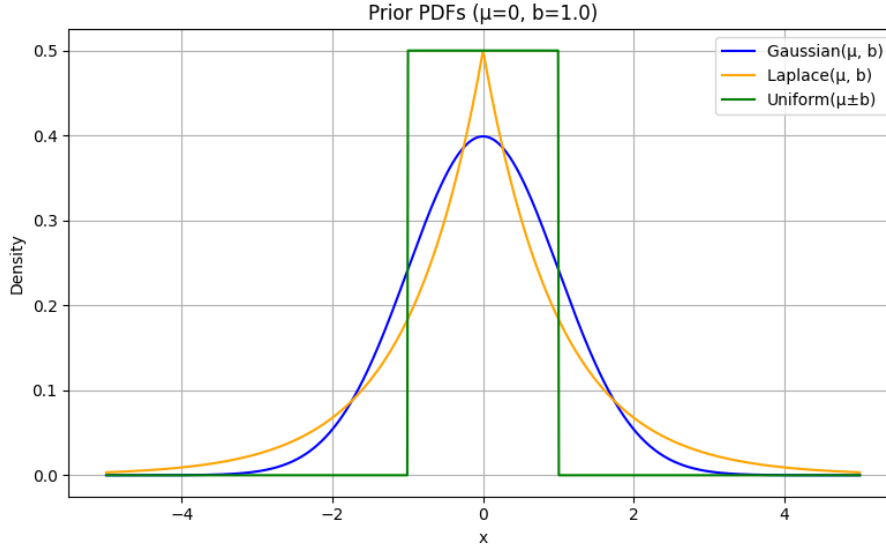
$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right) \quad (2.9) \quad f(x) = \frac{1}{2b} \exp\left(-\frac{|x-\mu|}{b}\right) \quad (2.10) \quad f(x) = \begin{cases} \frac{1}{b-a}, & x \in [a, b], \\ 0, & \text{otherwise,} \end{cases} \quad (2.11)$$

Based on the explanation for the three priors above, we can see that Gaussian and Laplace distribution have a similar parameter, which represents the location of the mean (μ) and the scale of how dispersed is the random variable of the parameters (σ in Gaussian, and b in Laplace). These paradigm are commonly used in Bayesian modelling modules like Pyro. The uniform distribution is parameterized differently compared to the other two distributions, as it is parameterized with a (lows) and b (highs). To have a similar sense accross all of the distributions, we introduced a parameter of *widths*, which is the difference between highs and lows, $widths = b - a$.

In this research, the two parameters for each distribution will be notated in a same notation, a and b . The table below shows the mapping of the parameters:

prior distribution	a (in this research)	b (in this research)
Gaussian	μ	σ
Laplace	μ	b
Uniform	a (lows)	highs – lows (widths)

Table 2.1: Prior distribution parameter used in this research

Figure 2.2: Prior Probability Distribution Function for $a = 0$ and $b = 1$ (Gaussian and Laplace) and $a = -0.5$ and $b=0.5$ (Uniform)

2.2.3 Bayesian Neural Networks

In this section, we will take a look at the concept of Bayesian Neural Networks. This research assumes that the reader is familiar with the basics of Neural Networks, including the concept of feed-forward and backpropagation. The idea of Bayesian Neural Networks (BNNs) dates back to some research done in the early 1990s. One of the early prominent works in Bayesian Neural Networks is the work done in [35]. In that research, although not explicitly mentioning Bayesian Neural Network (BNN), David J.C. Mackay proposes a quantitative and practical Bayesian framework for learning in feedforward networks. This research revolutionizes how to think about weight and biases by giving them a probabilistic interpretation within a learning model. This work criticized the backpropagation framework's limitations, such as arbitrary hyperparameter settings, high data demands, and a lack of uncertainty estimates.

To fix the issues on the above points, Mackay proposed a Bayesian way of doing backpropagation by the following:

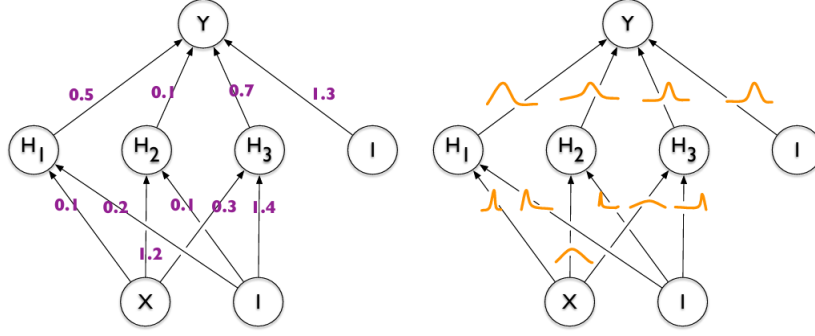
1. Objective setting of control parameters: In the Bayesian framework, model hyperparameters are determined by maximizing the marginal likelihood, which integrates over the parameter space with respect to the chosen priors. It automatically balances model fit and complexity, ensuring that the selected model achieves the best trade-off without overfitting.
2. Demand of data size: Bayesian Inference penalizes complexity automatically, as shown in Equation (2.12), by automatically trades off data fit vs model complexity without the need for a separate validation or test set to detect overfitting.
3. Lack of quantified uncertainty: Bayesian training infers a distribution over weights shown in Equation (2.13). The effective number of well-defined parameters (γ), are those weights, given the data, that are actually being used by the model in a meaningful way. With the above equation, well-defined parameters can be spotted by seeing which parameters have less variance, showing the model's confidence in their value.

$$P(\mathcal{D} | \mathcal{M}) = \int P(\mathcal{D} | \mathbf{w}, \mathcal{M}) P(\mathbf{w} | \mathcal{M}) d\mathbf{w} \quad p(\mathbf{w} | \mathcal{D}) \propto P(\mathcal{D} | \mathbf{w}) p(\mathbf{w}) \quad (2.13)$$

(2.12)

In [35], approximations are made to the posterior by using Laplace approximation and use Hessian of the loss to estimate posterior curvature. The term Bayesian Neural Network first coined in [39]. In that work, Radford M. Neal take the posterior distribution approximation to a new direction, by using Markov Chain Monte Carlo (MCMC) to not just do evidence maximization, but also sampling from the full posterior over weights, to approximate $p(\mathbf{w} \mid \mathcal{D})$. Illustration on how Bayesian Neural Networks differ to the Deterministic Neural Networks, in which each weight is assigned a distribution, is shown in Figure 2.3.

Figure 2.3: Illustration of Bayesian Neural Networks, as shown in [9]



2.2.4 Posterior Distribution Approximation

While Bayes' rule in Subsection 2.2.1 provide the base to compute posterior distribution over parameter or latent variables, in practice these posterior are often intractable to compute. The equations from (2.5) to (2.7) involve high-dimensional integrals that is computationally challenging and impossible to evaluate in complex models such as Bayesian Neural Networks.

To mitigate this issue, we use approximate inference methods, and one of them is **Variational Inference**. The method first introduced in [43], in which mean-field methods was introduced, to fit neural networks in minimizing a variational free-energy functional. Later work is done in [29] in employing mean-field variational inference in neural network training.

Instead of evaluating the true posterior $p_\theta(z|x)$ directly, Variational Inference (VI) introduces simpler tractable distributions $q_\phi(z)$ parameterized by ϕ . The goal is to select $q_\phi(z)$ that is closest to the true posterior according to a divergence measure, typically the **Kullback-Leibler (KL) divergence**:

$$\text{KL}(q_\phi(z) \parallel p_\theta(z|x)) = \mathbb{E}q_\phi(z) [\log q_\phi(z) - \log p_\theta(z|x)] \quad (2.14)$$

Since the true posterior is unknown, we optimize an equivalent and tractable objective called the **Evidence Lower Bound (ELBO)**. In maximizing ELBO, we ensure that the variational distribution $q_\phi(z)$ approximates the true posterior as close as possible, approximating the solution for Bayesian Inference in large-scale or high-dimensional models.

$$\text{ELBO} \equiv \mathbb{E}_{q_\phi(z)} [\log p_\theta(x, z) - \log q_\phi(z)] \quad (2.15)$$

When the dataset is large, computing ELBO over the entire dataset in each optimization step becomes expensive. **Stochastic Variational Inference (SVI)**, which was introduced in [30], combines VI with stochastic optimization. SVI subsamples mini-batches and computes estimate of the ELBO gradient.

Another method to approximate Bayesian Inference, besides Variational Inference, is **Markov Chain Monte Carlo (MCMC)**. First introduced in [37] with Metropolis algorithm, the work is extended in [28] to generalize the former into Metropolis-Hastings algorithm making MCMC more broadly applicable to Bayesian inference problems. This algorithm works by constructing a Markov Chain that generates a sequence of samples that approximate draws from the true posterior distribution.

2.3 Robust ML Systems

Robustness in Machine Learning (ML) systems refers as a system's ability to maintain accuracy and correct behaviour under both **security threats** and **reliability threats** [48].

1. Security threats

This type of threat comes from adversaries exploiting model weaknesses or implementation flaws. Common security threats are adversarial attacks, neural-level trojans, hardware attacks, and IP stealing. Adversarial attack work by adding imperceptible noise to the input, making the task of distinguishing clean and malignant inputs nearly impossible. This attack can cause misclassification, either by resulting in a random incorrect output class (untargeted attack), a specific designated output class (targeted attack), or simply reduce the confidence of the correct output class (confidence reduction). Some of the adversarial attack method are Fast Gradient Sign Method (FGSM) [24] and Jacobian saliency map approach [45].

2. Reliability threats

Even without active attackers, ML systems can fail due to hardware or environmental factors. Some types of reliability threat are intermittent faults, permanent faults, and environmental noise. Soft errors are a type of transient fault mostly caused by a radiation induced change of charge Q_{rad} in the transistors across the chip. If that charge exceed the threshold value of Q_{th} , the state of the transistor would change, resulting in a bit-flip. This phenomenon, which is called as Single Event Upset (SEU), will be explained in the next sub chapter.

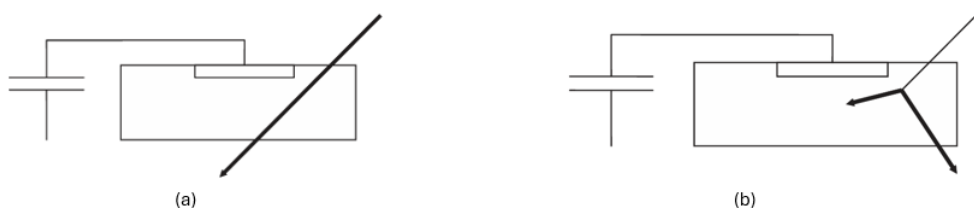
2.4 Single Event Effects

Single Event Effects (SEEs), as described in [42], happened through the action of a single ionizing particle as it penetrates sensitive nodes inside an electronic device. The effect of SEEs can be as small as a tiny glitch inside a monitor to a substantial error that is life threatening. On October 2008, a Qantas A330 flight from Singapore to Perth experienced a sudden and unexpected altitude change, resulting in 36 crew and passengers injured [3]. The Australian Transport Safety Bureau (ATSB) considers SEE as a possible cause of the accident [4]. Some variants of SEEs are Single Event Upsets (SEUs), Multibit Upsets (MBUs), and Single Event Burnouts (SEBs) [42].

Illustrations of heavy ions passing through a semiconductor is shown in Figure 2.4 (a) and (b). In Figure 2.4 (a), the ion creates a dense ionization track that generates enough charge to trigger a Single Event Upset (SEU). In Figure 2.4 (b), a proton causes a nuclear reaction inside the device, producing secondary heavy ions that deposit charge and may also result in an SEU.

SEUs or bit-flips have been simulated either by injecting permanent faults (non modifiable 0s or 1s) into the weights of CNN layers at the software level to assess reliability and classify prediction deviations [10], or by injecting transient single and double bit flips into GPU registers and instruction outputs at the architecture level to study application-level Silent Data Corruptions (SDCs) [27]. Both studies show that SEUs can lead to incorrect or unsafe outputs in Deep Neural Networks, potentially degrading their reliability in safety-critical applications. This research will follow a similar procedure to the SEU injection method in [27], in which we will inject SEU (bitflip) on the weight/bias parameter of the model **when it is doing inference on a new data**. An example of the impact of bitflips to stored weight/bias is shown in Table A.7.

Figure 2.4: Ionization path due to direct passage of heavy iron (a), and due to proton reaction in a device (b) as shown in [42]



The IEEE-754 standard ([11], [32]) defines binary (32, 64, 128 bits) and decimal (64, 128 bits) floating-point (FP) formats. Modern CPUs and frameworks (e.g. PyTorch) support binary machine numbers (FP32, FP64) for efficiency despite limited precision [17]. Since Pyro is built on PyTorch [16], this

research uses FP32 (Figure A.1), making it important to understand floating-point representation for analyzing the effects of Single Event Upsets (SEUs) on weight and bias parameters stored in FP32.

2.5 Robustness of Neural Networks

2.5.1 Robustness against Adversarial Attack

Before we take a look into robustness of neural networks against SEUs/bitflips, we will see what are the defenses against adversarial attack applied in recent research, especially for Bayesian Neural Networks.

Stochastic Gradient Langevin Dynamics (SGLD) [50] introduces noise in training to capture uncertainty, although it is not specifically designed for adversarial robustness. BNNs also have been reported to mitigate gradient-based attacks, e.g., Fast Gradient Sign Method (FGSM) and Projected Gradient Descent (PGD), due to posterior averaging [12]. **Spectral Expectation Bound Regularization (SEBR)** [53] also increases robustness against gradient-based attacks by regularizing the spectral norm of weights. Contradicting work includes [8], which shows that a more accurate posterior inference (e.g., Hamiltonian Monte Carlo, advanced VI) does not guarantee robustness.

2.5.2 Robustness against Weight Perturbation

Single Event Upset Induced Parameter Perturbation (SIPP) [51] shows that SEUs in memory or parameters can drastically degrade accuracy (from 70% to 1%). Proposed defense in this work includes Triple Modular Redundancy (TMR), Error-Correcting Codes (ECC), and Selective Protection on both.

Model-intrinsic robustness is explored in [15], where Weighted Gaussian (WG) and Rectified Weighted Gaussian (RWG) activation functions, smart pooling, dropout, and weight decay show improvements in deterministic CNNs under SEU-like perturbations.

While both of the works mentioned previously take an interest in the robustness of deterministic neural networks, especially variants of convolutional neural networks, against weight/bias perturbation, **no significant research is found on observing Bayesian Neural Networks (BNNs) robustness against weight perturbation yet.** This area of research is what this paper aims to look into.

2.6 Measure of Robustness

To evaluate how well a neural network withstands adversarial attack or weight/bias perturbation, we need a quantitative metric that can measure its robustness. This measure will help us in comparing the variation of models or strategies with each other and decide which ones are the most robust against perturbations. Different alternatives of robustness, both for input and parameter perturbations, are as follows:

1. Accuracy Delta

Accuracy delta is the most commonly used measure of robustness in this field of research. It measures the performance difference before/after perturbation. Smaller values are perceived to show a better robustness [15], [19], [40].

$$\Delta \text{Accuracy} = \text{Accuracy}_{after} - \text{Accuracy}_{before} \quad (2.16)$$

2. Difference in softmax

Softmax Difference [12] captures prediction confidence change. Robustness is $1 - \text{softmax difference}$. Later work [49] highlights the trade-off between accuracy and robustness.

$$\text{Softmax Difference} = \frac{1}{N} \sum_{j=1}^N \left| \langle f(\mathbf{x}_j, \mathbf{w}) \rangle_{p(\mathbf{w}|D)} - \langle f(\tilde{\mathbf{x}}_j, \mathbf{w}) \rangle_{p(\mathbf{w}|D)} \right|_{\infty} \quad (2.17)$$

While in general the measure of robustness is the same, some measure of robustness is exclusive to those aiming to minimize the impact of adversarial attacks. Other research includes a metric called Augmented Robustness, which was proposed in [18], that takes into account the "unknown" class if the output of the network does not achieve a certain threshold confidence.

Chapter 3

Design

3.1 Dataset Description

In this research, we are using an image dataset called "Ships in Satellite Imagery" [25], or "Shipsnet" in short. The dataset is satellite imagery captured from a commercial company called Planet. Those satellite images are taken from over the San Francisco Bay and San Pedro Bay areas of California. Initially, the pictures are orthorectified 3-meter pixel size, but for this task in detecting ships, 4000 80 x 80 RGB images are derived from it, labeled with either a "ship" (1) or "no-ship" (0) classification. Both classes are explained below:

1. Ship: This class includes 1000 images. To be qualified to be included in this class, ships are supposed to be centered in the image. The images consist of ships of different sizes, orientations, and atmospheric collection conditions.
2. No-ship: This class includes 3000 images.
 - (a) Land cover features, randomly sampled: buildings, bare earth, vegetation, water
 - (b) Images that only cover a portion of ships: these images, although they contain part of ships, do not qualify to be categorized as "ships"
 - (c) Images that were previously mislabeled by machine learning models, typically caused by bright pixels or strong linear features

Figure 3.1: Sample images from the Shipsnet dataset.



3.2 Deterministic CNN Model Architecture

The deterministic CNN model that we will set as a benchmark for this research is designed for binary image classification as observed from section 3.1. Here are the layers used, along with the input and output shape for each layer.

Layer	Input Shape	Output Shape	Trainable Weight	Trainable Bias
Conv1 + Pooling	[B,3,64,64]	[B,32,32,32]	864	32
Conv2 + Pooling	[B,32,32,32]	[B,64,16,16]	18,432	64
Flatten + Pooling	[B,64,16,16]	[B,16384]	32,768	2
Linear (Output)	[B,16384]	[B,2]	-	-

Table 3.1: Layers in our model

Conv1 and Conv2 represent the convolutional layer (`torch.nn.Conv2d`), followed by an activation function and a max pooling (`torch.nn.MaxPool2d`), then flattened to be processed in a fully connected layer. The training process of this model will use the Adam optimizer with a learning rate of 10^{-3} and a cross-entropy loss function. In this deterministic implementation of our model, with **52,162 trainable weights and biases**, do note that all the weights and biases for each neuron are fixed after the training is done. The model output is logits of the two classes, and the chosen class is the argmax of these logits, which is the class with the highest logits among all.

3.3 Bayesian Implementation of the Deterministic CNN Model

The Bayesian CNN model designed in this experiment preserves the core structure of the previously mentioned deterministic CNN in subsection 3.2. The first thing that is different from the deterministic CNN is how each weight and bias parameter is modeled as a random variable drawn from a prior distribution. As already mentioned in subsection 2.2.2, we will be using the Gaussian prior, Laplace prior, and Uniform prior with parameters a and b .

Bayesian Inference for these random variables will be done with **Stochastic Variational Inference (SVI)**. The variational family used in this experiment is the **mean-field assumption**. In this type of variational family, all weights and biases are assumed to be independent.

3.4 Training Process

The loss function used to train the Bayesian CNN model is the negative Evidence Lower Bound (ELBO). While the generic version of it mentioned in Equation 2.15 is used in generic Variational Inference, the BNN-specific ELBO that we will use in this experiment is defined as:

$$\text{ELBO} \equiv \mathbb{E}_{q_\phi(W)} [\log p(D | W)] - \text{KL}(q_\phi(W) \parallel p(W)) \quad (3.1)$$

Given the dataset D with observed inputs and labels, the ELBO loss aims to maximize the expected log-likelihood of the data from the sampled network weights while minimizing the KL divergence between the learned variational distribution $q(W)$ and the prior $p(W)$, trying to balance the data fit and regularization. This exact formula is the loss function that can be found in the Pyro package in Python, specifically in `pyro.infer.trace_elbo`.

The optimizer used in this experiment is **ClippedAdam**, taken from the Pyro Python package, with a learning rate of 10^{-3} . The default setting of weight decay is 0, but it can be adjusted for the purpose of understanding the impact on the overall robustness.

3.5 Inference Process

Inference for the Bayesian CNN model, in this experiment, is done in a Monte Carlo Sampling fashion, as shown in Algorithm 3.1.

With the algorithm below, the default Monte Carlo samples per batch is set to be equal to 10. The number is arbitrary and can be adjusted depending on the needs, with a higher number will take from more posterior weight samples. Each iteration samples new weights ($W^{(s)}$) from a variational posterior family. Logits are averaged before applying argmax, reducing prediction variance. This mechanism shows the aggregation of Bayesian predictive uncertainty, which can not be found in the deterministic CNN.

Input: Trained Bayesian model f , guide $q_\phi(W)$, data loader $\mathcal{D}$, number of samples $S = 10$

Output: Predicted class labels $\hat{y}$ for all inputs

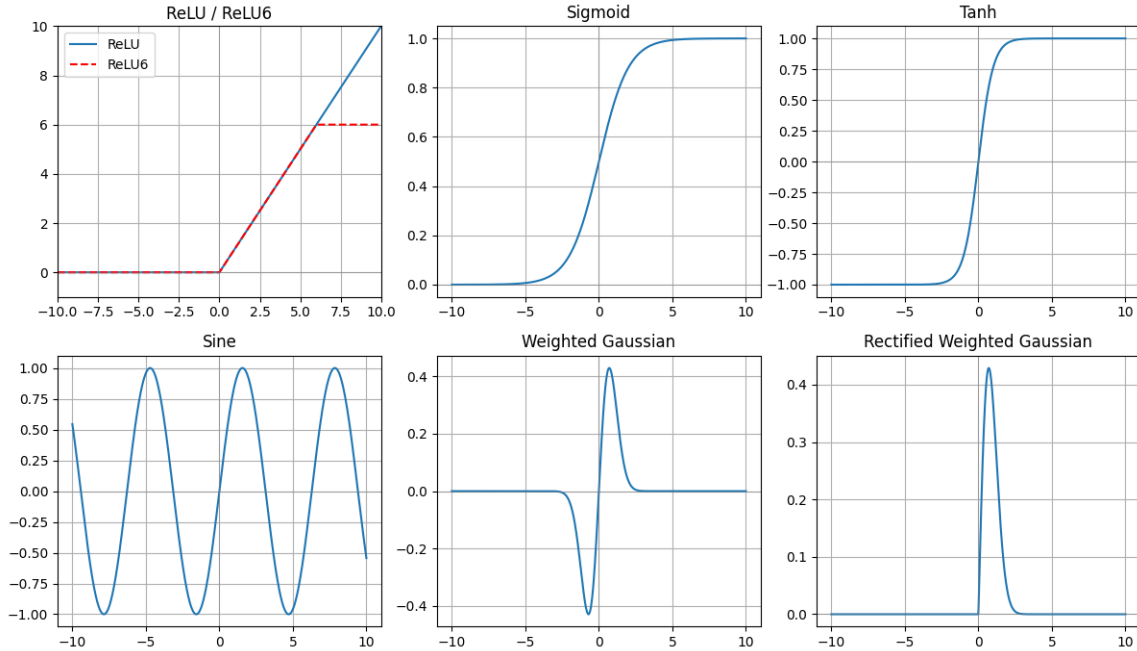
```

foreach mini-batch  $(x, y)$  in  $\mathcal{D}$  do
  Initialize logits accumulator:  $L \leftarrow 0$ 
  for  $s = 1$  to  $S$  do
    Sample weights:  $W^{(s)} \sim q_\phi(W)$ 
    Compute logits:  $\ell^{(s)} \leftarrow f(x; W^{(s)})$ 
    Accumulate:  $L \leftarrow L + \ell^{(s)}$ 
  end
  Compute mean logits:  $\bar{L} = \frac{L}{S}$ 
  Predicted class:  $\hat{y} = \arg \max(\bar{L})$ 
  Store  $\hat{y}$ 
end
return all predicted labels  $\hat{y}$ 

```

Algorithm 3.1: Monte Carlo Sampling for Bayesian CNN Inference

Figure 3.2: Activation Functions Plot



3.6 Activation Function Variations

In this research, we will vary the activation functions to form different model variations. We have 7 activation functions that we want to compare in terms of their performance, which some widely known activation functions: ReLU, Sigmoid, Tanh, and Sine. We also want to test the Weighted Gaussian (WG) and Rectified Weighted Gaussian introduced in [15] in this experiment, which are shown in Equation (3.2) and Equation (3.3). As previous research shows that ReLU is the least robust activation function, we also want to look at a different version of it called ReLU6, which is shown in Equation (3.4). The plots of each activation function are shown in Figure 3.2.

$$\text{WG}(x) = xe^{-x^2} \quad (3.2)$$

$$\text{RWG}(x) = \max(0, xe^{-x^2}) \quad (3.3)$$

$$\text{ReLU6}(x) = \min(\max(0, x), 6) \quad (3.4)$$

3.7 Prior Distribution Parameter Variations

In this research, for each prior distribution that we want to test, as mentioned in sub-section 3.3, we will vary the parameter of the distribution to see the effect on model accuracy and SEU robustness. We will

fix the value of a (μ or $lows$) in our prior to be equal to 0, as we have no strong belief about the exact value that best fits the model, but we still want to have a prior distribution that is pleasantly centered. We will vary the value of b with the following:

1. 0.1: very narrow prior that shows we are supposedly confident that the weights are deviating only slightly from 0. As the weight is kept in a small window, this discourages flexibility in the model weight, which leads to underfitting.
2. 1: a moderate prior that is commonly used. This value is not expected to be underfitting nor overfitting.
3. 10: very wide prior that shows we have weak belief towards the true value. The model is too free to fit the data, which poses the risk of overfitting.

3.8 Model Variations

To observe different model architectures and their impact to the robustness of our Bayesian CNN model, we will explore a few options that we can choose to apply to that model, on top of our base model.

1. SmartPooling

In [47], the author evaluates how single radiation-induced soft errors in GPUs could propagate through CNN layers and impact detection accuracy. They propose mechanisms like ECC (error-correcting codes) and ABFT (algorithm-based fault tolerance) for matrix multiplication operations. In the same work, they also propose a modified maxpooling layer called "**Reliable Maxpooling**". This layer detects unreasonable values, typically chosen in maxpooling layers, and either halts frame processing or propagates the second-largest value to reduce error propagation.

Examination of how reliable maxpooling is working in a weight perturbed condition, [15] implemented reliable maxpooling in the name of SmartPooling, in which the proposed layer can be a useful mitigation for unbounded activation functions like ReLU. Their work shows that 16% robustness is achieved in Relu-based deterministic CNNs equipped with SmartPooling.

In this research, we typically set the value of Threshold $\theta = 10$. In activating this SmartPooling, we replace every MaxPooling that was applied in our paper with it. The smartpooling algorithm is shown in Algorithm 3.2.

2. Dropout layer

Work in [15] also explored how the dropout layer can benefit from mitigating the SEU-induced performance drop. In this work, the author explored two dropout probability p values, which are $p = 0$ (off state) and $p = 0.5$. (on state). Dropout layer is activated during training, which hopefully will help the model regularize against SEU by deactivating random neurons during training, so the network cannot rely too much on any single pathway, forcing redundancy and better generalization.

3. Weight decay

Weight decay is a common regularization technique in CNNs to prevent overfitting. Regularization allows complex models to be trained on data sets of limited size without severe over-fitting [7]. The motivation in modifying the weight decay to inspect the impact of SEU/bitflips lies in the ability of weight decay to constrain the weights, hopefully reducing the impact of bitflips on any of the weights/bias. In our deterministic CNN model, we will use the Adam optimizer from PyTorch [33], and the Bayesian implementation will use the ClippedAdam optimizer from Pyro [44]. Updated weight parameter, considering weight decay, is formulated in the equation below:

$$w_{t+1} = w_t - \eta \left(\nabla L(w_t) + \lambda w_t \right) \quad (3.5)$$

In that formula, w_t represents the weight parameter at step t , η is the learning rate, $\nabla L(w_t)$ is the gradient of the loss with respect to the weights, and λ is the weight decay coefficient. In this research, weight decay is set in two alternative values, which are $\lambda = 0$ and $\lambda = 10^{-4}$.

These model variations, added with the base model, will make a total of 4 model variations, which will be explained more in Subsection 4.4.

3.9 SEU process

The process of simulating SEU in this research is done with the help of Python's `struct` module. Here are some rules that are imposed during the simulation of SEU injection:

Input: Input tensor $X \in \mathbb{R}^{N \times C \times H \times W}$;
 Kernel size k ; Stride s ; Threshold θ ;
Output: Pooled tensor $Y \in \mathbb{R}^{N \times C \times H' \times W'}$

1. Extract all sliding windows (patches) of size $k \times k$ from X with stride s , producing:

$$P = \text{Unfold}(X) \quad \text{where } P \in \mathbb{R}^{N \times C \times k^2 \times L}$$

where L is the number of windows per feature map.

2. For each patch $P_{n,c}$, compute the top-2 largest values:

$$(\text{max1}, \text{max2}) = \text{TopK}(P_{n,c}, k = 2)$$

where max1 is the largest and max2 is the second-largest value in each window.

3. Detect abnormal activations (spikes):

$$S_{n,c,l} = \begin{cases} 1 & \text{if } \text{max1}_{n,c,l} > \theta, \\ 0 & \text{otherwise} \end{cases}$$

for every batch n , channel c , and window index l .

4. If the algorithm is set to only see the spiked values without altering the output, set:

$$Y_{n,c,l} = \text{max1}_{n,c,l}$$

If the reliable maxpooling is used to overwrite the spiked value, replace detected spikes with the second-largest value:

$$Y_{n,c,l} = \begin{cases} \text{max2}_{n,c,l}, & S_{n,c,l} = 1 \\ \text{max1}_{n,c,l}, & S_{n,c,l} = 0 \end{cases}$$

5. Reshape Y back to spatial dimensions:

$$Y \in \mathbb{R}^{N \times C \times H' \times W'}, \quad H' = \frac{H}{k}, \quad W' = \frac{W}{k}$$

return Y

Algorithm 3.2: SmartPool: Reliable Max-Pooling with Outlier Detection and Correction

1. SEU is done before the weight/bias is sampled. Thus, the SEU process is attacking the parameter of either a or b from the guide $q_\phi(W)$. The weight/bias is sampled from the newly perturbed guide $q'_\phi(W)$. Example of sampled weight distribution before and after SEU is shown in Figure A.2
2. Single Event Upset/bitflip is attacking only one of the bit locations within the 32-bit IEEE 754 representation at a time.
3. SEU is bitflipping a 0 value to 1, or 1 value to 0. Hence, it's bitflipping the value into another value regardless of the starting value of the bit.
4. SEU is only done in a single neuron's weight/bias at a time.
5. SEU might change the value of any of the distribution of the parameter into a different sign (+ to -, or vice versa). The value of parameter b , which is the variance of the Gaussian/Laplace distribution, and the width of the Uniform distribution is not expected to be negative. In this experiment, we assume that we does not enforce any constraint to keep the value b positive after the SEU, that may make the model unable to return a valid prediction.

In this research, for our deterministic CNN models, we will do 2,352 SEU injections. This SEU injections are the combination of 28 different models that we have (4 model variations and 7 activation functions), with each model get 84 different SEU injections, which comprises a combination of 2 neuron attack locations (first/last neuron), 3 layer (conv1/conv2/fc1), 2 layer parameter (weight/bias), and 7 bit index.

For Bayesian CNN models, 28 different models will be attacked with 468 different SEU injections, 252 (7×36) for the prior center and 216 (6×36) for the prior spread, for a total of 13,104 SEU injections.

The attack comprises of 36 different attacks, which combination comprises of 3 prior (Gaussian, Laplace, Uniform), 2 neuron attack locations, 3 layers and 2 layer parameters. We can only attack 6 variations of bit index into the prior spread, as hitting the bit index 0 for prior spread will change the prior spread parameter sign from + to -, which is the limitation of our prior as mentioned in Subsection 2.2.2.

3.10 Robustness Measure

The robustness measure that we will use in this research will be based on the two metrics from Section 2.6, which are Accuracy Delta and Softmax Difference. Accuracy of the model after the SEU injection does not necessarily lower than the accuracy before the SEU injection is done, and we want to look both for positive or negative change of accuracy and penalize them equally, without cancelling each other. To do that, we will modify the formula of Accuracy Delta shown in Equation 2.16 to be the one in Equation 3.6 below.

$$\text{Absolute Accuracy Difference} = |\text{Accuracy}_{after} - \text{Accuracy}_{before}| \quad (3.6)$$

Our research questions rely on comparing multiple model variations and deciding which one is the best based on the given result. While both robustness measures are useful in achieving our goals on its own, interpreting both of the robustness measure at the same time may result in confusion, e.g., one model might have a low absolute accuracy difference, but a high softmax difference. To make the model comparison easier, we devise a new aggregate robustness measure called the Aggregate Robustness Index (ARIn), which takes Equation (3.6) and Equation (2.17).

$$\text{ARIn} = \sqrt{\frac{(\text{Absolute Accuracy Difference})^2 + (\text{Softmax Difference})^2}{2}} \quad (3.7)$$

Chapter 4

Implementation

4.1 Experimental Environment

For this project, we use `Python 3.11` as the main programming language to do every tasks, from preparing the dataset, training and testing deterministic and Bayesian CNN models, up to the simulation of SEU/bitflips and its analysis. `Anaconda 24.7.1` is used in this project to provide package and virtual environment management. For deep learning tasks, especially the deterministic CNN models, we use `PyTorch 2.7.1`, `torchaudio 2.7.1`, and `torchvision 0.22.1`. All of these modules are installed with support for `Cuda 12.8`, enabling GPU acceleration.

For the implementation of Bayesian CNN models, we chose `Pyro 1.9.1`. Pyro is a universal probabilistic programming language (PPL) built in Python and powered by PyTorch as its backend [6]. It provides a flexible and expressive framework for constructing deep probabilistic models, combining the strengths of modern deep learning with Bayesian inference techniques. GitHub is used in this project for the purpose of code versioning.

4.2 Computation Resource

This project is run on the Elastic Compute Cloud (EC2) instance on Amazon Web Services (AWS), with credits provided by the University of Bristol. Specifically, this project is using the G4dn EC2 instance family. This instance family is equipped with NVIDIA T4 GPU and custom Intel Cascade Lake CPUs [1]. The instance type chosen is the G4dn.xlarge, with specification as follows [2]: 1 GPU, 16 GiB of GPU memory, 4 vCPU, 16 GiB memory, and 1 x 125 GB NVMe SSD storage. The instance is operated on the Ubuntu 22.04 operating system on top of AWS Deep Learning Base GPU AMI.

4.3 Dataset Preparation

The dataset mentioned in Section 3.1 is then preprocessed with `torchvision.transforms` before we train them to our models:

1. **Resize** : Image is resized from 80 x 80 pixels to 64 x 64 pixels. This will help us in fewer computation needed, and convenience of using image with a power of two that will make it easier for downsampling with pooling layers.
2. **Scaling** : From the image with RGB channels that have a value of intensity between 0 and 255, the value is converted to be ranged from 0 to 1.
3. **Standardization** : For each channel, we calculate the mean and standard deviation of pixel density, then normalize each values with:

$$x_{\text{norm}} = \frac{x - \mu}{\sigma} \quad (4.1)$$

This will ensure that the value of each pixel ranges from [-1,1] and centered around 0. This will help our deterministic and Bayesian CNN models in learning, especially as optimizers assumes the input to be centered and scaled similarly.

4.4 Training of Deterministic CNN Model

Model is trained in 20 epochs and saved each time best validation accuracy is reached. Model names are coded as follows:

1. Model 0 : This is the base model as mentioned in Section 3.2
2. Model 1 : Instead of using `torch.nn.MaxPool2d`, this model uses smartpooling, as mentioned in subsection 3.8 point 1.
3. Model 2 : Dropout layer (`torch.nn.Dropout`) is applied after conv2 layer with $p = 0.5$
4. Model 3 : Weight decay is implemented to the Adam optimizer with $\lambda = 10^{-4}$

4.5 Training of Bayesian CNN Model

The model trained in the Bayesian CNN setting have the similar architecture to the one in the deterministic setting. Model is trained in 100 epochs, model was saved each time best train accuracy is reached. Model names follows the nomenclature of the models trained in the deterministic CNN model (subsection 4.4). Model is trained on three different b parameter : $b = [0.1, 1, 10]$.

4.6 Implementations in Pyro

Here are some adjustments made in the Bayesian CNN models in Python by utilizing Pyro:

1. `pyro.nn.module.PyroModule` is used to embed probabilistic constructs (sampling, parameters with priors, constraints) directly into the `torch.nn.Module` paradigm. In our case, it transforms `torch.nn.conv2d` and `torch.nn.Linear` into Bayesian layers, by enabling their weights and biases to be treated as random variables rather than fixed variables.
2. `pyro.nn.module.PyroSample` is used as a descriptor inside `PyroModule` to define random variables that will be sampled from a prior distribution. For each layer's weight and bias we define each from what prior distribution is used for each. In this research, we define 3 distribution types : Gaussian, Laplace, and Uniform inside this descriptor, along with each parameter of a and b .
3. The **BNN model** and its prior distribution has been created by the two classes above. For Stochastic Variational Inference (SVI) to work, we need the parameterized approximation to the posterior distribution over the parameters sampled from `pyro.nn.module.PyroSample`, which is the **guide**. To automatically initialize the guide, we use `pyro.infer.autoguide.guides.AutoNormal`. This automatic guide creates a separate Normal/Gaussian distribution (with learnable mean and scale/variance) for each `PyroSample` site. In this implementation, each weight and bias is assumed to be **independent** with each other.
4. While `pyro.infer.autoguide.guides.AutoNormal` can be used for the Gaussian Prior, it is not suitable for the Laplace and Gaussian prior, as we want to design our research to have a posterior approximation similar to the shape of our prior. Pyro implementation for automatic guide for Laplace and Uniform is not provided by the Pyro module; therefore, in this research, we modify `AutoNormal` to match the Laplace and Uniform distribution. In this research, we are naming them `AutoLaplace` and `AutoUniform`.
5. `pyro.infer.svi.step` is used to do the training step in SVI for our Bayesian CNN model instead of `torch.optim.Adam.step` in deterministic CNN. The loss value returned is the negative ELBO for that batch.
6. Learned posterior from `Autonormal`, e.g. `Autonormal.locs.conv1.weight`, is stored inside an object called `pyro.get_param_store()`. To ensure no parameter leak is happening in-between experiments, `pyro.clear_param_store()` is used to clear all global parameters.
7. `pyro.poutine.trace(guide).get_trace(images)` samples the weight from the guide distribution, and saving the sampled deterministic value to a `trace`.
8. `pyro.poutine.replay(model, trace)` is then used to inject the sampled weights from the earlier to the model. This makes the model uses them deterministically in the forward pass process.

Chapter 5

Critical Evaluation

5.1 Analysis of Model Performance Metric

In this section, we will monitor how each model, before SEU injection, performs on a given test set. The performance metric used in this experiment is test accuracy, which was mentioned in Section 2.6. For deterministic CNN model, we will compare all model variants across all activation functions, compare the results, and decide which model variant and activation function are the best compared to others. For the Bayesian CNN model, in addition to that, we will also take a look at prior b parameter variations, and see which one yields in a better result.

5.1.1 Training Result of Deterministic CNN

Model Variant	RWG	WG	relu	relu6	sigmoid	sin	tanh	average
Model 0	.9838	.9750	.9762	.9712	.9700	.9825	.9712	.9757
Model 1	.9838	.9712	.9788	.9788	.9700	.9712	.9725	.9752
Model 2	.9825	.9788	.9788	.9800	.9638	.9762	.9738	.9763
Model 3	.9812	.9738	.9738	.9775	.9675	.9800	.9700	.9748
Average	.9823	.9747	.9769	.9769	.9678	.9775	.9719	.9755

Table 5.1: Test accuracy of different activation functions across model variants in evaluating Deterministic CNN Models

Based on our test set, it is seen that the Rectified Weighted Gaussian is the best activation function for our Bayesian CNN Model. Model 2 is observed to result in a better test accuracy compared to all other model variants.

5.1.2 Training Result of Bayesian CNN

Summary of test accuracy for Bayesian CNN Models, to find the best activation function and model variant, is provided in Table 5.2 and Table 5.3. The full result of test accuracy for each model variant, activation, prior, and prior b are shown in Table A.4 A.5 and A.6.

Based on the test accuracy result, there are some highlights that we can make about the result. First, for models with prior $b = 0.1$ (Table A.4), relu6 yields better test accuracy compared to other activation function. Across all models, Gaussian and Laplace prior have a test accuracy around 75%, while Uniform prior results in test accuracy around 90%. Best accuracy among all models achieved by model 0, with average test accuracy of 0.8039. Second, with prior $b = 1$ (Table A.5), relu dominantly yields better test accuracy compared to other activation function. Best accuracy among all models achieved by model 1, with average test accuracy of 0.8332. Lastly, with prior $b = 10$ (Table A.6), tanh is observed to have a significantly better test accuracy across almost all prior and activation function. Best accuracy among all models achieved by model 3, with average test accuracy of 0.8117.

While this research does not necessarily demand the model with the best accuracy to be the focus of our SEU injection process, we will proceed the SEU injection process to focus on all the priors with prior_b = 1, as shown in Table 5.3 that also shows Model 1 is the best model among others. Its also

observed from our experiment that across all model variations that we have, relu activation function shows the best average test accuracy, as shown in Table 5.2.

Prior b	RWG	WG	relu	relu6	sigmoid	sin	tanh	average
0.1	.8058	.7859	.8270	.8249	.7788	.8034	.7909	.8024
1	.7618	.7788	.9123	.8671	.7892	.8071	.8832	.8285
10	.7540	.7674	.8618	.8090	.8218	.7489	.8851	.8068
Average	.7739	.7774	.8670	.8336	.7966	.7865	.8531	.8126

Table 5.2: Average test accuracy (mean across models) by prior and activation function. Best per prior b in bold.

Model Variant	RWG	WG	relu	relu6	sigmoid	sin	tanh	average
Model 0	.7733	.7817	.9046	.8700	.7979	.7958	.8783	.8288
Model 1	.7800	.7762	.9125	.8750	.7979	.8054	.8854	.8332
Model 2	.7471	.7796	.9117	.8875	.7746	.8171	.8754	.8276
Model 3	.7467	.7775	.9204	.8358	.7862	.8100	.8938	.8243
Average	.7618	.7787	.9123	.8671	.7892	.8071	.8832	.8285

Table 5.3: Average test accuracy by model for prior $b = 1$. Best per model variant in bold.

A finding on training the Bayesian CNN model with uniform prior

ELBO loss, which was explained in Equation (2.15), includes the joint probability $p_\theta(x, z)$, which can be break down in Equation (5.1). The ELBO loss becomes Equation (5.2).

$$\log p_\theta(x, z) = \log p(x|z) + \log p(z) \quad (5.1) \quad \text{ELBO} \equiv \mathbb{E}_{q_\phi(z)} [\log p(x|z) + \log p(z) - \log q_\phi(z)] \quad (5.2)$$

As stated in Equation (2.11), the value of $p(z)$ depends on the bound of the uniform prior. For the value of $a = 0$ and $b = 1$, in this research, the value of $p(z)$ is stated as Equation (5.3). As the variational guide (in this case, the **AutoUniform**) samples a z value such that $z \notin [0, 1]$, the $\log p(z)$ becomes $-\infty$, making the ELBO loss from Equation (5.2) becomes as Equation (5.4).

$$\log p(z) = \begin{cases} \log(0.5), & \text{if } z \in [0, 1] \\ \log(0) = -\infty, & \text{otherwise} \end{cases} \quad (5.3) \quad \text{ELBO} = \mathbb{E}_{q_\phi(z)} [\log p(x|z) - \infty - \log q_\phi(z)] = -\infty \quad (5.4)$$

The epoch of which the training process start to experience $-\infty$ varies between activation functions and model variants, but an observable phenomenon can be seen across the variation of prior b values. For $b = 0.1$, as the range for the sampled weight can go beyond $[0, 0.1]$ quick, the first epoch already encounters the $-\infty$ ELBO loss. For $b = 1$, it's observed that most models can't get beyond 10 epoch, while for $b = 10$ can go to epoch 60 to 70, or even finish the training in 100 epochs.

As the ELBO loss becomes $-\infty$, the process of the optimizer in updating the guide parameter breaks because of failing backpropagation process due to gradients that become ∞ or **NaN**. One way to prevent this from happening is by applying clipping mechanism to keep the sampled weights in the bound of the uniform prior. For this research we want to see the effect of SEU bitflips to the result of robustness score. Therefore, clipping is intentionally avoided to allow out-of-bound values to show their effects.

5.1.3 Test Accuracy comparison between Deterministic CNN Model and Bayesian CNN model

Based on our findings in this chapter, **Bayesian CNN model is showing an inferior performance in test accuracy** compared to the deterministic CNN model. This observation is consistent across all activation functions, model variants, and Bayesian model activation function. The average accuracy of test data that can be achieved by the deterministic CNN model is 97.55%, while the Bayesian CNN model can only achieve 82.85% for prior b value equal to 1. it's suspected that the performance difference is

inherent to whether we are implementing Bayesian or deterministic CNN model. Here are some factors that can possibly result in this observation:

1. All weights and bias in the Bayesian CNN neural network are initialized with a similar prior parameter (a and b), assuming that all weight and bias' distributions are following the same distribution.
2. Optimization of Bayesian Neural Networks with ELBO loss is harder compared to cross-entropy in deterministic CNN model. KL divergence in ELBO loss can pull the variational posterior distribution away from the optimal point to stay closer to the prior (posterior collapse [34]).

5.2 Analysis of Performance Against SEU

In this section, most table will provide the aggregate value of SEU robustness measure by grouping the experiment result based on several criteria (e.g. model variation, activation function) and then averaging them. For simplicity, we denote these aggregated metrics as Average Absolute Accuracy Difference (AAAD) and Average Softmax Difference (ASD). Aggregated Robustness Index (ARIn) will be calculated based on these two values.

5.2.1 Experiment Output

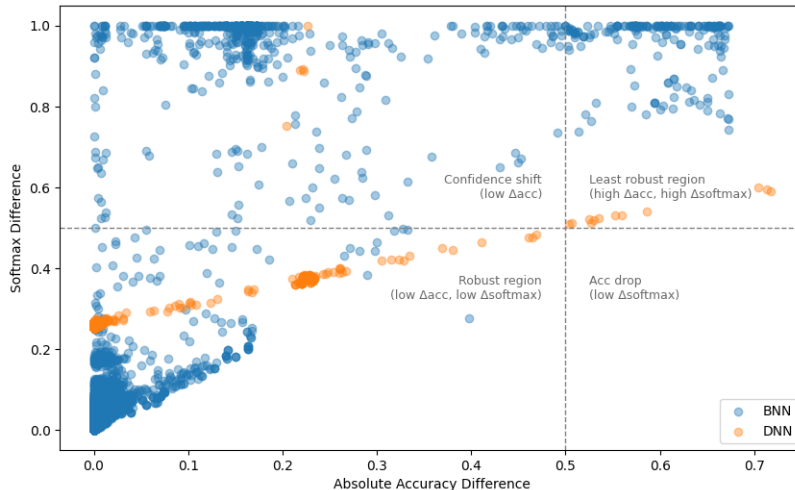
For each SEU injection, the following metrics are recorded:

1. Accuracy difference : This metric is calculated as mentioned in Equation (2.16). Then, we compute the Absolute Accuracy Difference as shown in Equation (3.6).
2. Softmax difference, as mentioned in Equation (2.17).
3. Absolute difference : This is calculated by subtracting the newly observed value after SEU with the original value before SEU.

In this section, the robustness analysis primarily relies on two metrics: the absolute accuracy difference and the softmax difference. In addition, we introduce our newly defined Aggregate Robustness Index (ARIn), as formalized in Equation (3.7), which will be reported alongside these metrics when results are presented in tabular form. For most visualizations, robustness will be illustrated using scatter plots, where the x-axis represents the absolute accuracy difference and the y-axis represents the softmax difference. This representation is intended to convey the intuition that models located in the bottom left region of the plot exhibit greater robustness, whereas those in the top right region demonstrate the least robustness. The visualization approach is inspired by the work of [12], with modifications, most notably the use of softmax difference rather than $1 - \text{softmax}$ on the y axis.

The main visual cue used in these scatter plots will be mostly rely on colors, to show the different categories of our experiment result and how they manifested in the SEU robustness. An example of the aforementioned figure, that shows the unaggregated result of SEU injection Robustness, colored by the type of model used (Bayesian or deterministic), is shown in Figure 5.1 below.

Figure 5.1: Robustness of Deterministic and Bayesian CNN model



Model Type	AAAD	ASD	ARIn
BNN	.0244	.1074	.0779
DNN	.0149	.2696	.1909

Table 5.4: Summary of average absolute accuracy difference and average softmax difference Deterministic vs Bayesian CNN

Robustness of Deterministic CNN models

From the aforementioned figure, we can infer that the impact of SEU on Absolute Accuracy Difference and Softmax Difference exhibits an approximately linear relationship, showing a clear correlation between the two performance metrics. This observation is similar from what has been observed from [12], in which the author spotted a trade-off between pure accuracy and robustness (1 - softmax difference) for high performing deterministic networks against adversarial attack (FGSM).

Robustness of Bayesian CNN models

Contrary to the Deterministic CNN model SEU injection result that has a clear correlation between the two performance metric in a linear straight line, the performance shown by the Bayesian CNN model does not exhibit a same pattern. In Bayesian CNN Model SEU Injection from the figure above, some noticeable points make up a linear correlation of absolute accuracy difference and softmax difference, but there are also observations that does not support this relationship. Some SEU Injection results show high softmax difference, while still keeping the absolute accuracy difference low.

Robustness of Deterministic CNN models vs Bayesian CNN Models

Based on Table 5.4 it is observed that the deterministic CNN model is showing more robustness against SEU injection in terms of absolute accuracy difference. It shows that the average absolute accuracy difference if a deterministic CNN model is attacked by SEU injection is 1.49%, while the Bayesian CNN model is expected to have an absolute accuracy difference of 2.44%.

This phenomenon can be triggered by the way that Bayesian Neural Network is sampling the weight/bias distribution parameter from the guide. In deterministic CNN model, the SEU only change one fixed weight/bias. In Bayesian CNN Model, as the distribution parameter is perturbed by the SEU, all Monte Carlo (MC) samples are drawn from the same perturbed distribution parameter. This makes every MC sample to be perturbed in the same way. This, will in turn disturb the average logits that is needed to classify the data with softmax. The resulting condition would be the data that is more prone to misclassification due to disturbed sampled weight and bias.

On the other hand, in terms of average softmax difference, Bayesian CNN model can be seen to demonstrate a better robustness against SEU injection. Softmax difference, which indicates the change in the predicted class probability, is observed to be 0.2696 on the Deterministic CNN Model, and 0.1074 on the Bayesian CNN Model. This means that Bayesian CNN Model has a smaller shift in output probabilities compared to the Deterministic CNN Model.

The observed condition can happen due to how logits are calculated after MC sampling. As we run over the network 10 times, or as much as the number of samples is determined, logit shifts from the first run might be diluted by the subsequent run, as we are taking the average of all 10 runs to be the input of the softmax, that will classify our data. Although the chance of misclassifying some data sample is increased, Bayesian CNN model offer a more stable prediction probability against SEU.

To conclude our comparison, we employ the Aggregate Robustness Measure (ARIn) defined in Equation (3.7). Based on this metric, we find that **Bayesian Neural Network exhibits greater robustness than the Deterministic Neural Network**, as evidenced by its smaller ARIn value (0.0779 compared to 0.1909 for the latter). In the applications where class confidence or predictive probability constitutes a critical factor, the Bayesian CNN model is therefore more suitable to use compared to the deterministic CNN model. Some applications can include medical imaging (e.g. If the model uncertainty exceeds the threshold, the case is "rejected" or flagged for human evaluation [20]) and ensemble modeling (in which multiple models are combined to produce aggregate predictions [5]).

Possible explanation on the "high softmax difference but low accuracy difference" on the Bayesian CNN

The Bayesian CNN models demonstrate superior robustness in terms of softmax difference under SEU injection. This observation is supported by the scatter plot in Figure 5.1, where a substantial number of SEU-injected Bayesian CNN results are located in the bottom-left region on the plot. This positioning indicates lower absolute accuracy differences and lower softmax differences, in contrast to the nearly linear distribution exhibited by the deterministic model. In addition, a considerable number of Bayesian CNN results appear in the top-left region of the plot, characterized by **high softmax differences but relatively low accuracy differences**, further distinguishing their behavior from that of the deterministic CNN.

This observation arises from the Monte Carlo sampling process during inference (Algorithm 3.1, with number of samples = 10). Under SEU injection, the variability of the logits across MC samples increases, while the mean logits before and after injection largely unchanged. Consequently, the nonlinear softmax function produces distributions with higher variability, which may lead to noticeable changes in softmax differences without altering the predicted class. This explains why many Bayesian CNN results fall in the bottom-left region of the scatter plot (low absolute accuracy difference and low softmax difference). In contrast, the deterministic CNN generates only a single fixed logit vector, so SEU injection affects the prediction only when the perturbation is sufficient to push the logits across a decision boundary, producing the near-linear distribution observed in the plot.

To conclude, these results reflect the role of uncertainty in the Bayesian CNN under SEU injection. During inference, the averaging process across Monte Carlo samples can either smooth out the effect of a bitflip (low absolute accuracy difference and low softmax difference), or amplify it, resulting in a higher softmax difference while accuracy remains unchanged, as observed in the top-left of the plot. Our findings indicate that the smoothing effect dominates, which explains why the majority of Bayesian CNN SEU injection outcomes cluster in the bottom-left region.

5.2.2 Robustness by model variation

Model Variant	AAAD		ASD		ARIn	
	DNN	BNN	DNN	BNN	DNN	BNN
Model 0	.0145	.0243	.2694	.1106	.1908	.0800
Model 1	.0142	.0254	.2699	.1109	.1911	.0805
Model 2	.0157	.0221	.2702	.0944	.1914	.0685
Model 3	.0152	.0257	.2687	.1138	.1903	.0825

Table 5.5: Robustness of Deterministic and Bayesian CNN model by model variant

Robustness of Deterministic CNN models

Figure A.3 shows the performance metric of each SEU injection for Deterministic CNN Model, colored by model variant, without any aggregation. Table 5.5 summarizes these results by averaging the Absolute Accuracy Difference (AAAD) and Softmax Difference (ASD) across model variants. Based on the Aggregate Robustness Index (ARIn), **Model 3 (trained with weight decay) demonstrates the strongest robustness**. Although it exhibits slightly worse AAAD compared to the base model (Model 0), it achieves a noticeably lower ASD. This is consistent with the expectation that weight decay reduces the magnitude of the weights, leading to a smaller logits and hence smaller softmax differences.

In comparison to prior work [15], our findings diverge. While it suggests that a variant equivalent to our Model 2 achieved the best AAAD, our experiments indicate that Model 2 has the least robust AAAD, while Model 1 (with smartpooling) performs best in AAAD.

Robustness of Bayesian CNN models

Figure A.4 shows the performance metrics of SEU injections for the Bayesian CNN models, with points colored by model variant. As shown in Table 5.5, **Model 2 (trained with dropout) is the most robust among the Bayesian variants**, exhibiting the lowest values for both AAAD and ASD. Although AAAD is generally similar across Bayesian variants, the reduced ASD of Model 2 makes its particularly notable. This outcome aligns with findings in [15], despite differences in experimental setup.

On the robustness of Bayesian CNN model trained with dropout layer

The significant robustness of Model 2 compared to other model variants in the Bayesian setting can be attributed by the nature of Dropout layers that demonstrate redundancy during training. As some neurons are inactive during training, our neural network can not just too reliant on any single neuron to carry critical information/infer a feature. In our Bayesian CNN model without dropout layer, there might be a chance that a SEU/bitflips can hit a dominant neuron and shift our prediction significantly. With a regularized model with dropout during training, the resulting model has no dominant neuron, so during inference, other neurons are as contributing as others, resulting in less shift in prediction if one of the neurons is being hit by bitflip.

5.2.3 Robustness by prior and prior parameter

Prior Prior Parameter	AAAD		ASD		ARIn	
	Center	Spread	Center	Spread	Center	Spread
Gaussian	.0177	.0293	.0794	.1281	.0575	.0929
Laplace	.0193	.0287	.0823	.1306	.0598	.0946
Uniform	.0231	.0303	.1054	.1277	.0763	.0928

Table 5.6: Robustness of Bayesian CNN model by prior and prior parameter

In this section, we will only discuss the findings from SEU attack on Bayesian CNN models, as the deterministic CNN models does not have prior and prior parameter in their architecture. Table 5.6 aggregates the absolute accuracy difference and softmax difference of our experiment, grouping them by the prior distribution (Gaussian, Laplace, and Uniform) and the prior parameters, which are center (*a*) and spread (*b*).

Which prior is the most robust?

For the absolute accuracy difference, both Gaussian and Laplace prior have a similar result on the center and spread, with the Uniform distribution is the worst among all prior distributions. In terms of softmax difference, the Gaussian and Laplace distribution also performs similarly on the prior center and are both better compared to the Uniform distribution. A different observation can be found on the spread, which shows the Uniform and Gaussian perform similarly and better compared to the Laplace distribution. Overall, we can conclude that the **Gaussian prior is the most robust prior** among all, with Laplace prior shown a better accuracy robustness on prior spread attack, and the uniform distribution shown a better softmax robustness on prior spread attack.

Gaussian prior vs Laplace prior

As shown in Figure 2.2, each prior distribution have a different Probability Distribution Function (PDFs). The Gaussian distribution has lighter tails compared to the Laplace distribution with the same parameter. As we shift the mean for both distribution, we also pull the tail samples toward or across the decision boundary. Because Laplace distribution have a heavier tail, the probability of tail samples to get sampled is higher, thus increasing the likelihood of prediction changes. The Uniform distribution is unsurprisingly doing worst, as the probability density of Uniform distribution is flat, making the samples evenly spread across the specified range. A shift in *a* value for uniform distribution move a larger fraction of samples across the decision boundary, with the same probability of being sampled, making the prediction accuracy worse.

Laplace distribution is less affected in absolute accuracy difference if the spread parameter is being attacked compared to the Gaussian distribution, due to the fact that Gaussian distribution is more concentrated in the center. If we increase or do bitflip on the scale parameter for Gaussian distribution, the distribution of sample near the center widen and possibly move the sampled weight/bias across the decision boundary. For the Laplace distribution, modifying the scale parameter does not change the sampled value much, as the probability density of Laplace is already wide relative to the Gaussian prior.

Uniform prior robustness in softmax difference against the spread SEU injection

Since changing the scale parameter for the Gaussian and Laplace prior affect the peak height and decay rate in the tails, if we do Monte Carlo sampling for weight/bias sampling, the difference of sample values is expected to be higher compared to the Uniform prior that have a flat PDF. This is due to the fact that the high-density center is now less dense and mass shifts into the tails, which will amplify the variability in logits. The uniform distribution can yield a better softmax difference, due to the fact that the each value of weight/bias have a similar probability to be sampled. A slight change in Uniform’s scale parameter, for a latent weight x , will result in a smaller change of probability, compared to the change for the same latent weight x in Gaussian/Laplace prior.

On the robustness of prior center vs prior spread against SEU

Based on the Table 5.6 above, we can also retrieve a conclusion about how the SEU interacts with the center prior parameter and spread prior parameter. For all metrics, **the center prior parameter is consistently more robust than the spread prior parameter**. Robustness of Bayesian Neural Networks against SEU injection on the prior center parameter comes from the nature of shifting the mean of the prior that moves all the weight samples by the same absolute amount, preserving the relative positions and allowing them to cancel out during the Monte Carlo averaging. Shift in the spread parameter will change the weight sampled to be significantly different from the mean (either pushing some further into the tails or other closer to the center), making the resulting logits to be much different than before the SEU injection happened.

5.2.4 Robustness by activation function

Activation Function	AAAD		ASD		ARIn	
	DNN	BNN	DNN	BNN	DNN	BNN
RWG	.0078	.0168	.2651	.0958	.1875	.0688
WG	.0178	.0213	.2716	.0964	.1924	.0698
relu	.0234	.0379	.2835	.1626	.2012	.1180
relu6	.0114	.0248	.2653	.1012	.1878	.0737
sigmoid	.0136	.0190	.2598	.0800	.1839	.0581
sin	.0141	.0294	.2702	.1056	.1913	.0775
tanh	.0163	.0214	.2714	.1102	.1923	.0794

Table 5.7: Robustness of Deterministic and Bayesian CNN model by activation function

Based on Table 5.7, we can observe that the Rectified Weighted Gaussian activation function has the most robust absolute accuracy difference among all activation function for the deterministic CNN model (which has been proven in [15]) and Bayesian CNN Model. On our newly introduced softmax difference measure, the sigmoid activation function achieves the lowest Average Softmax Difference (ASD) in both settings.

Relu, which is the most expressive activation function that we have in this experiment (as seen from Figure 3.2), shown a comparably better test accuracy (Table A.4) despite showing the worst robustness. RWG, which yields the lowest test accuracy, achieves the strongest robustness, especially in terms of absolute accuracy difference. This observation shown us a symptom of a **trade-off between test accuracy and SEU robustness**, which is also observed in the work of [12].

It can also be seen from Table 5.7 that the sigmoid activation function have the second best robustness in absolute accuracy difference on BNNs, while also having the best softmax difference. In terms of SEU, which is a change of weight and bias before activation function is implemented, let’s get back on how the neuron is calculated in the feed-forward process. This process, as mentioned in [7] is notated as:

$$a_j = \sum_{i=1}^D w_{ji}^{(1)} x_i + w_{j0}^{(1)} \quad (5.5)$$

where $x_1, x_2, \dots, x_D$ are inputs, $j = 1, 2, \dots, M$ and the superscript (1) represents the parameters in the first ‘layer’ of the network, which have a ‘width’ of M . $w_{j0}^{(1)}$ is Each of a_j is then transformed in the following equation:

$$z_j = h(a_j) \quad (5.6)$$

where $h(\cdot)$ is a differentiable activation function.

$w_{ji}^{(1)}$ which represents weight, or $w_{j0}^{(1)}$ which represents bias, are the component in this feed-forward process that is prone to the SEU injection. As one of both values changed into SEU injected value a'_j , we would like to know how this change can effect the value of z into z'_j , the lower the change, the difference in softmax value will be also lower.

In this experiment, the weight/bias is expected to be near zero with our setting of prior parameter $a = 0$ and $b = 1$. The majority of bitflips that we do will result in a change of weight/bias that is near zero, when we hit bit index that is not 0 (sign bit) and 1 (first exponent bit). Therefore, let's see the impact of changing a to z by observing the differential of our activation function $h'(\cdot)$. We will take a look into sigmoid and the seemingly similarly shaped tanh, and its differential near 0.

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \frac{d\sigma(z)}{dz} = \sigma(z)(1 - \sigma(z)), \quad \sigma(0) = 0.5 \Rightarrow \sigma'(0) = 0.25$$

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}, \quad \frac{d}{dz} \tanh(z) = 1 - \tanh^2(z), \quad \tanh(0) = 0 \Rightarrow \tanh'(0) = 1$$

From our calculation of differential for both activation functions above, we can see that the differential on sigmoid at 0 is 0.25, while the tanh is at 1. A slight change in input for tanh have a bigger shift in the value after activation function. Therefore, we can conclude that the sigmoid function gained robustness in SEU because of a smaller gradient near $z = 0$.

It will also be interesting to look at the gradient of our activation function on extreme values, especially in looking in the effect of SEU injection on bit index 1 (first exponent). [7] mentioned that the sigmoid decay asymptotically like e^{-z} for $z \rightarrow +\infty$, while tanh, which has $\tanh'(z) = 1 - \tanh^2(z)$, will decay in $4e^{-2z}$ for $z \rightarrow +\infty$, which is faster than the sigmoid. We can assume that the tanh function will dampen the z value better on the extreme values.

5.2.5 Robustness by layer and layer parameter

Layer	Layer Parameter	AAAD		ASD		ARIn	
		DNN	BNN	DNN	BNN	DNN	BNN
conv1	bias	.0048	.0095	.2651	.0641	.1875	.0458
conv1	weight	.0070	.0129	.2696	.0762	.1907	.0546
conv2	bias	.0021	.0089	.2622	.0723	.1854	.0515
conv2	weight	.0058	.0136	.2640	.0987	.1868	.0704
fc1	bias	.0326	.0459	.2773	.1720	.1974	.1259
fc1	weight	.0372	.0555	.2792	.1611	.1991	.1205

Table 5.8: Robustness of Deterministic and Bayesian CNN model by layer and layer parameter

Table 5.8 summarizes the SEU Injection result by averaging the value of Absolute Accuracy Difference and Softmax Difference across layers and layer parameter of the CNN architecture. Based on that table, it is observed that convolutional layer 2 is the most robust, followed by convolutional layer 1. The fully connected layer is the least robust, as it exhibits the highest mean of softmax difference and the highest mean absolute accuracy difference. These findings also align with the result shown in [15], where the further convolutional layer (conv2 and conv3) have the lowest average test delta and the fully connected layer has the highest.

Layer SEU Robustness, propagation depth

Fully connected layer, which is the furthest layer we have in our architecture, is the least robust due to how the SEU injection impacts the final logits. If we do the SEU injection early in the convolutional 1 layers, there are multiple occassion that nonlinear process will dampen the effect of SEU.

First, every convolution layer will be followed by an activation function. Nonlinear activation function can help reduce the impact of SEU or bitflips. After the activation function, the result is taken into pooling layer, in which the multiple values can be aggregated into a smaller dimension. If the pooling layer is utilizing the average pooling layer, the value of the SEU will be averaged with other values, reducing the impact. In our case study, we are using max pooling. The value of the SEU might just be propagated to the next layer if the resulting value is the highest among all. Therefore, as the work in [15] suggested, smart pooling is used, to remove values that are above specified threshold, especially if the value was hit by SEU.

Let's compare this condition with fully connected layer from our model architecture. In this layer, the SEU injected values will not undergo any non-linear transformation like what we had in the convolutional layers. Therefore, this value will directly impact the calculation of logits and class probability, thus explaining the higher absolute accuracy difference and softmax difference.

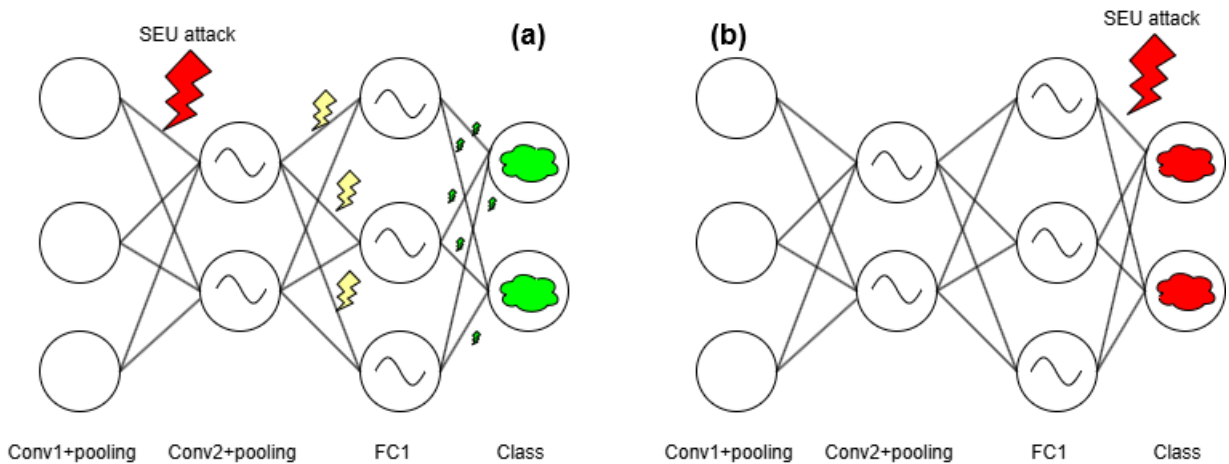
Figure 5.2 illustrates how the effects of an SEU injection propagate through a CNN model. In panel (a), an SEU injected into an early convolutional layer (indicated by the red lightning bolt) undergoes attenuation as it passes through subsequent layers. This weakening is shown by yellow and then smaller green lightning symbols in later layers, reflecting the diminishing influence of the fault. The sinusoidal symbols within the neurons represent non-linear transformations, which further dampen the perturbation. Consequently, the effect on the final logits is minimal, represented by the green cloud at the output.

In contrast, panel (b) shows an SEU injected directly into the fully connected (FC1) layer. Here, the perturbation is not mitigated by earlier non-linear transformations and instead propagates strongly to the output layer, producing the most severe distortion in the logits, indicated by the red clouds.

Layer component SEU Robustness, weight vs bias

Based on the result of our experiment, we can see that the weight is less robust compared to the bias, when we inject them with SEU/bitflips. This is due to the nature of the neural networks itself. Looking back at Equation (5.5), we can see that the weight is multiplied by the input value x_i to compute a_j . **Change in weight to the value of a_j is amplified by the x_i ,** compared to the change in bias that is merely a constant.

Figure 5.2: Illustration of SEU injection and its propagated effect on conv1 layer (a) and fc1 layer (b) in a CNN model



5.2.6 Robustness by attacked bit location

Bit Index	Average Log Absolute Difference		AAAD		ASD		ARIn	
	DNN	BNN	DNN	BNN	DNN	BNN	DNN	BNN
0	-1.1804	-0.7909	.0005	.0075	.2613	.0533	.1848	.0381
1	37.0504	37.8829	.1023	.1206	.3193	.4080	.2371	.3009
3	-1.4815	-0.6454	.0002	.0070	.2612	.0527	.1847	.0376
6	-0.5849	-0.4291	.0013	.0070	.2617	.0529	.1850	.0378
10	-2.2308	-1.3945	.0001	.0067	.2611	.0525	.1847	.0374
15	-3.7360	-2.8997	.0000	.0067	.2611	.0525	.1847	.0374
21	-5.5422	-4.7058	.0000	.0067	.2611	.0528	.1847	.0376

Table 5.9: Robustness of Deterministic and Bayesian CNN model by attacked bit location

To analyze the effect of SEU injections at different bit positions, we compute the log absolute difference of weight or bias values before and after an SEU, defined as:

$$\text{Log Absolute Difference} = \log_{10}(|\text{value after SEU} - \text{value before SEU}|) \quad (5.7)$$

In this experiment, the attacked bit index in this experiment are sign (index 0), exponent (index 1,3,6), and mantissa (index 10, 15, 21). As shown in Table 5.9, flipping bit index 1 yields an average log absolute difference of approximately 37 and results in the worst robustness across all metrics (AAAD, ASD, and ARIn).

By contrast, perturbations to the mantissa bits have negligible impact on AAAD, with values close to zero, and produce robustness metrics nearly identical to those from bit index 3 in the exponent. Attacks on bit index 6 also yield slightly higher AAAD than the mantissa bits, through still minor compared to bit index 1.

Notably, no clear correlation can be established between the log absolute difference and robustness metrics such as AAAD or ASD. For example, bit index 0 (sign bit) produces a moderate log absolute difference yet yields slightly higher ASD compared to mantissa flips. Overall, these results confirm that SEUs affecting the exponent, particularly at bit index 1, are the most harmful to model robustness.

5.2.7 Robustness by original bit attacked

Original Bit Value	AAAD		ASD		ARIn	
	DNN	BNN	DNN	BNN	DNN	BNN
0	.0275	.0455	.2767	.1734	.1966	.1268
1	.0002	.0069	.2612	.0529	.1847	.0378

Table 5.10: Robustness of Deterministic and Bayesian CNN model by original bit attacked

In this subsection, we also compare how the robustness is affected by the original bit value/state before the SEU. Table 5.10 shows that bitflip from 1 to 0 exhibits an extremely more robust model compared to bitflip from 0 to 1, especially on the absolute accuracy difference. It's not aligned with the claim from [26] that stated the opposite.

This difference can be explained by the distribution of weights and biases in our models, which are typically close to 0 or 1. Consequently, the original bit value/state play an important role in determining the impact of SEUs. For instance, in all SEU injections involving bit index 1, the original bit value/state was always 0. Since Subsection 5.2.6 already established that bit index 1 is highly vulnerable to SEUs, the observed lack of robustness in bitflip from 0 to 1 are consistent with these findings.

Chapter 6

Conclusion

The purpose of this research is to examine the robustness of Bayesian Neural Networks designs against Single Event Upsets. The research starts by training both deterministic and Bayesian CNN Models, followed by SEU Injection on different model variants for each setting. In this chapter, we will reflect on our findings, the limitations encountered in this research, and suggestions of future work.

6.1 Project Status

The research aims of this dissertation, as stated in Subsection 1.3.1 are as follows:

1. Train multiple deterministic and Bayesian Neural Networks with different design choice and note its performance
2. Do multiple experiment of SEU injection on the different deterministic and Bayesian Neural Network design and note its performance
3. Compare the robustness of each different design
4. Validate whether Bayesian Neural Networks are more robust compared to deterministic Neural Networks.

Training result

Across all model variants and activation functions, **deterministic CNN model results in a better model performance**, which is test accuracy. The average test accuracy for Deterministic CNN model is 97.55% and Bayesian CNN model only achieve 82.85% for prior b value equal to 1. This observation is suspected to happen due to the simplification of our Bayesian CNN model design in assuming that all weights and biases are initialized with the same prior and prior parameter. The other factor that can also contribute to this examination is due to the fact that KL Divergence, which is part of the ELBO loss used to train deterministic CNN model, is harder to optimize than cross entropy which is used in the deterministic CNN model.

For all model, activation, and prior variation, our **Bayesian CNN model with prior parameter b equal to 1 results in a better test accuracy** of 82.85% compared to the prior $b = 0.1$ models that have the average test accuracy of 80.24% and $b = 10$ of 80.68%. The observed activation function that is better is either relu, relu6, or tanh.

Robustness of different model designs against SEU Injection

In this research, we test multiple model designs and compare its robustness for both deterministic CNN model and Bayesian CNN model. Here are the results as stated in Subsection 5.2.

1. **Model variations:** Dropout during training enhanced robustness against SEU in Bayesian CNNs, with model 2 showing the best both absolute accuracy difference and softmax difference. Weight decay (model 3) contributed to reduced softmax sensitivity in deterministic CNN, while the model with the best absolute accuracy difference is model 1 (model with smartpooling).

2. **Prior and prior parameter:** The Gaussian prior is found to be the most robust, while Laplace and Uniform each showed better robustness under spread perturbations for absolute accuracy difference and softmax difference, compared to other priors, respectively. Across all priors, the center parameter proved to be more robust than the spread parameter. This is due to the fact that shifting the mean preserves relative sample positions, while changes in spread increase variability in logits.
3. **Activation functions:** Among all activation functions, Rectified Weighted Gaussian is the most robust in terms of absolute accuracy difference. The phenomenon of trade-off between test accuracy (which relies on the expressiveness of activation function) and SEU robustness is observed, as seen in the Relu activation function. Relu6, which is the bounded version of relu, shows better robustness against SEU injection. Sigmoid activation function shown better robustness in terms of softmax difference and ARIn.
4. **Layer and layer parameter:** Convolutional layers are observed to be more robust to SEU than the fully connected layer, as non-linear activations and pooling partially dampen perturbations before reaching the logits. On the other hand, SEUs in the fully connected layer directly affect the logits, resulting in larger accuracy and softmax difference. Additionally, weights are less robust than biases, since weight perturbations are amplified by the inputs, while biases act only as additive shifts.
5. **Attacked bit location:** SEU attacks on the first exponent bits caused the worst robustness quality, while the mantissa bit flips had a minimum effect.
6. **Original bit attacked:** Flipping a bit from 1 to 0 is found to be far more robust, especially in terms of accuracy, which is not aligned with a finding from previous research. The result is influenced by the fact that most weights and biases are closer to 0 or 1, meaning that the original bit state strongly determine the effect of SEU. In particular, first exponent bit flips, which is the most devastating bit flips, are impacting 0 bit values on all occasions, which explain the result that we have.

Is Bayesian Neural Networks are more robust against deterministic Neural Networks against SEU injection?

To conclude, our research question on whether Bayesian CNN model is more robust than the deterministic CNN model can be answered with Table 5.4. Both model variation have their own strength, with deterministic CNN model that have better average absolute accuracy difference of 1.49% compared to Bayesian CNN model that has 2.44%. The Bayesian CNN Model is more robust in softmax difference (0.1074) against the deterministic CNN model (0.2686). Our newly defined Aggregate Robustness Index (ARIn), which aggregates both measures, shows that **Bayesian CNN model (0.0779) is more robust compared to deterministic CNN model (0.1909)**. Bayesian CNN model implementation benefits the scenario of which our Neural Networks takes class confidence/probability as one of the major aspect of its implementation.

6.2 Limitations

The ELBO loss used in the Bayesian CNN models training consists of the expected log-likelihood and KL divergence. In implementation, Pyro’s `pyro.infer.trace_elbo` does not expose them separately, preventing direct observation of their individual changes during training. This is especially useful if we want to modify the KL divergence term that went to $-\infty$ for uniform prior in Subsection 5.1.2.

This research assumes simplified priors with the same parameter across all weights and biases. To address this matter that results in lower performance compared to the deterministic CNN model, we could employ a warm start technique, in which we train the CNN models in the deterministic way first and use the resulting weight/bias as a starting point for our Bayesian CNN model.

This work also assumes that all weights and biases do not correlate/depends with each other, which is the main assumption in mean-field variational guide. If memory and storage permits, instead of utilizing Pyro’s `AutoNormal`, we can use `AutoMultivariateNormal` to have an automatic guide that also stores a Cholesky factorization, which covariance matrix can capture the correlations between latent variables.

While this research has covered 84 different Bayesian CNN models that each cover 168 different SEU injection occurrences, the SEU injection we conducted with single bit-flips at a time, which may understate the real-world condition, in which Multi Bit Upset (MBU) can happen.

6.3 Future Work

The CNN model architecture used in this experiment is quite small (2 convolutional layers and 1 fully connected layer with 52,162 parameters) on a dataset with 2 classes (Shipsnet). Dataset with more classes will offer a new perspective on how the softmax values are not symmetric, while a more complex architecture (e.g. ResNet, AlexNet, EfficientNet) resemble a more likely real-world application of models deployed in harsh environments.

Testing our Bayesian Neural Networks with more variation of prior distribution is also a promising area of research. One of an interesting prior distribution to try on is the horseshoe prior, which is claimed to express robustness at handling unknown sparsity and large outlying signals [13].

Designing a Bayesian CNN model with another activation function also open up a new possibility of finding a more robust model than what we have in this research. Although the experiment setting is on the robustness against adversarial attack, work on [14], stated that Parametric Activation Functions like Parametric Shifted Sigmoidal Linear Unit (PSSiLU) shows more robustness when combined with adversarial training.

Previous researches has mixed opinion on how Markov Chain Monte Carlo (MCMC) method can be more robust against adversarial attack against the deterministic model. Our research only uses Stochastic Variational Inference (SVI) from Pyro module, therefore it will be also compelling to find out how MCMC methods like Hamiltonian Monte Carlo (HMC) or No-U-Turn Sampler (NUTS) [31] performs, despite the increase in inference complexity.

A simpler approximate inference strategies can also be explored, which can include MC Dropout [21] and Stochastic Weight Averaging-Gaussian (SWAG) [36]. In doing so, we can see how these simpler methods keeping their performance against SEU injection and how the complexity in terms of model and training time can be saved.

Bibliography

- [1] Amazon EC2 G4 Instances — aws.amazon.com. <https://aws.amazon.com/ec2/instance-types/g4/>. [Accessed 03-08-2025].
- [2] Cloud Compute Instances – Amazon EC2 Instance Types – AWS — aws.amazon.com. <https://aws.amazon.com/ec2/instance-types/>. [Accessed 03-08-2025].
- [3] Airsafe. Serious injuries on a qantas a330 on 7 october 2008. <https://www.airsafe.com/plane-crash/qantas-flight-72-airbus-a330-learmonth.htm>. Accessed: 2025-07-26.
- [4] Australian Transport Safety Bureau. Interim factual report no. 2: In-flight upset involving airbus a330-303, vh-qpa, 154 km west of learmonth, wa on 7 october 2008. Technical report, Australian Transport Safety Bureau, Canberra, ACT, Australia, November 2009. Aviation Occurrence Investigation AO-2008-070.
- [5] D. Barber and Christopher M. Bishop. Ensemble learning in bayesian neural networks. In *Generalization in Neural Networks and Machine Learning*, pages 215–237, United Kingdom, January 1998. Springer.
- [6] Eli Bingham, Jonathan P. Chen, Martin Jankowiak, Fritz Obermeyer, Neeraj Pradhan, Theofanis Karaletsos, Rohit Singh, Paul Szerlip, Paul Horsfall, and Noah D. Goodman. Pyro: Deep Universal Probabilistic Programming. *Journal of Machine Learning Research*, 2018.
- [7] Christopher M. Bishop. *Pattern Recognition and Machine Learning (Information Science and Statistics)*. Springer, 1 edition, 2007.
- [8] A.C. Blaas. *On the adversarial robustness of Bayesian machine learning models*. PhD thesis, University of Oxford, 2021. PhD thesis.
- [9] Charles Blundell, Julien Cornebise, Koray Kavukcuoglu, and Daan Wierstra. Weight uncertainty in neural networks. In *Proceedings of the 32nd International Conference on International Conference on Machine Learning - Volume 37*, ICML’15, page 1613–1622. JMLR.org, 2015.
- [10] A. Bosio, P. Bernardi, A. Ruospo, and E. Sanchez. A reliability analysis of a deep neural network. In *2019 IEEE Latin American Test Symposium (LATS)*, pages 1–6, March 2019.
- [11] Richard L. Burden and J. Douglas Faires. *Numerical Analysis*. Brooks/Cole, Cengage Learning, Boston, MA, ninth edition, 2010.
- [12] Ginevra Carbone, Matthew Wicker, Luca Laurenti, Andrea Patane, Luca Bortolussi, and Guido Sanguinetti. Robustness of bayesian neural networks to gradient-based attacks. *CoRR*, abs/2002.04359, 2020.
- [13] Carlos M. Carvalho, Nicholas G. Polson, and James G. Scott. Handling sparsity via the horseshoe. In David van Dyk and Max Welling, editors, *Proceedings of the Twelfth International Conference on Artificial Intelligence and Statistics*, volume 5 of *Proceedings of Machine Learning Research*, pages 73–80, Hilton Clearwater Beach Resort, Clearwater Beach, Florida USA, 16–18 Apr 2009. PMLR.
- [14] Sihui Dai, Saeed Mahloujifar, and Prateek Mittal. Parameterizing activation functions for adversarial robustness. *CoRR*, abs/2110.05626, 2021.

-
- [15] William Dennis and James Pope. A framework for developing robust machine learning models in harsh environments: A review of cnn design choices. In *Proceedings of the 17th International Conference on Agents and Artificial Intelligence - Volume 2: ICAART*, pages 322–333. INSTICC, SciTePress, 2025.
 - [16] Pyro Developers. Introduction to pyro — pyro tutorials 1.9.1 documentation. https://pyro.ai/examples/intro_long.html, 2025. [Accessed 28-07-2025].
 - [17] PyTorch Developers. Numerical accuracy — pytorch 2.7 documentation. https://pytorch.org/docs/stable/notes/numerical_accuracy.html, 2025. [Accessed 28-07-2025].
 - [18] Wasim Essbai, Andrea Bombarda, Silvia Bonfanti, and Angelo Gargantini. A framework for including uncertainty in robustness evaluation of bayesian neural network classifiers. In *2024 IEEE/ACM International Workshop on Deep Learning for Testing and Testing for Deep Learning (DeepTest)*, pages 25–32, 2024.
 - [19] Yunzhen Feng, Tim G. J. Rudner, Nikolaos Tsilivis, and Julia Kempe. Attacking bayes: On the adversarial robustness of bayesian neural networks, 2024.
 - [20] Matteo Ferrante, Tommaso Boccato, and Nicola Toschi. Enabling uncertainty estimation in neural networks through weight perturbation for improved alzheimer’s disease classification. *Frontiers in Neuroinformatics*, Volume 18 - 2024, 2024.
 - [21] Yarin Gal and Zoubin Ghahramani. Dropout as a bayesian approximation: Representing model uncertainty in deep learning. In Maria Florina Balcan and Kilian Q. Weinberger, editors, *Proceedings of The 33rd International Conference on Machine Learning*, volume 48 of *Proceedings of Machine Learning Research*, pages 1050–1059, New York, New York, USA, 20–22 Jun 2016. PMLR.
 - [22] Andrew Gelman, John B. Carlin, Hal S. Stern, David B. Dunson, Akti Vehtari, and Donald B. Rubin. *Bayesian Data Analysis*. Chapman Hall/CRC Texts in Statistical Science Series. CRC, Boca Raton, Florida, third edition, 2013.
 - [23] Gianluca Giuffrida, Lorenzo Diana, Francesco de Gioia, Gionata Benelli, Gabriele Meoni, Massimiliano Donati, and Luca Fanucci. CloudScout: A deep neural network for on-board cloud detection on hyperspectral images. *Remote Sens. (Basel)*, 12(14):2205, July 2020.
 - [24] Ian J. Goodfellow, Jonathon Shlens, and Christian Szegedy. Explaining and harnessing adversarial examples, 2014. cite arxiv:1412.6572.
 - [25] Robert Hammell. Ships in satellite imagery. <https://www.kaggle.com/dsv/61115>, 2018.
 - [26] Muhammad Abdullah Hanif, Faiq Khalid, Rachmad Vidya Wicaksana Putra, Mohammad Taghi Teimoori, Florian Kriebel, Jeff (Jun) Zhang, Kang Liu, Semeen Rehman, Theodoros Theodoridis, Alessandro Artusi, Siddharth Garg, and Muhammad Shafique. *Robust Computing for Machine Learning-Based Systems*, pages 479–503. Springer International Publishing, Cham, 2021.
 - [27] Siva Kumar Sastry Hari, Timothy Tsai, Mark Stephenson, Stephen W. Keckler, and Joel Emer. SASSIFI: An architecture-level fault injection tool for gpu application resilience evaluation. In *2017 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 249–258. IEEE, 2017.
 - [28] W. K. Hastings. Monte carlo sampling methods using markov chains and their applications. *Biometrika*, 57(1):97–109, 04 1970.
 - [29] Geoffrey E. Hinton and Drew van Camp. Keeping the neural networks simple by minimizing the description length of the weights. In *Proceedings of the Sixth Annual Conference on Computational Learning Theory, COLT '93*, page 5–13, New York, NY, USA, 1993. Association for Computing Machinery.
 - [30] Matthew D. Hoffman, David M. Blei, Chong Wang, and John Paisley. Stochastic variational inference. *J. Mach. Learn. Res.*, 14(1):1303–1347, May 2013.
 - [31] Matthew D. Homan and Andrew Gelman. The no-u-turn sampler: adaptively setting path lengths in hamiltonian monte carlo. *J. Mach. Learn. Res.*, 15(1):1593–1623, January 2014.
-

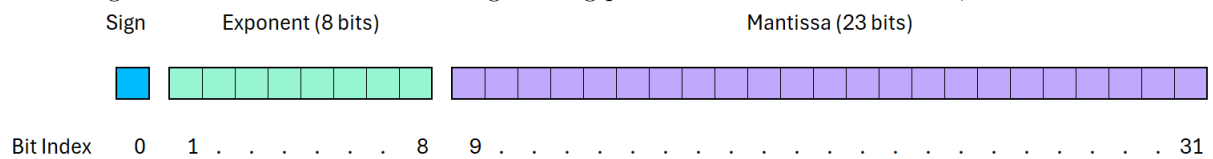
- [32] IEEE Computer Society. IEEE Standard for Floating-Point Arithmetic. IEEE Std 754-2019, July 2019. Published on 2019-07-22.
- [33] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- [34] James Lucas, George Tucker, Roger B. Grosse, and Mohammad Norouzi. Don’t blame the elbo! A linear VAE perspective on posterior collapse. *CoRR*, abs/1911.02469, 2019.
- [35] David J. C. MacKay. A practical bayesian framework for backpropagation networks. *Neural Comput.*, 4(3):448–472, May 1992.
- [36] Wesley J. Maddox, Timur Garipov, Pavel Izmailov, Dmitry Vetrov, and Andrew Gordon Wilson. *A simple baseline for Bayesian uncertainty in deep learning*. Curran Associates Inc., Red Hook, NY, USA, 2019.
- [37] N. Metropolis, A. W. Rosenbluth, M. N. Rosenbluth, A. H. Teller, and E. Teller. Equation of state calculations by fast computing machines. *J. Chem. Phys.*, 21:1087–1092, 1953.
- [38] Kevin P. Murphy. *Machine Learning: A Probabilistic Perspective*. The MIT Press, 2012.
- [39] Radford M. Neal. *Bayesian Learning for Neural Networks*. Springer-Verlag, Berlin, Heidelberg, 1996.
- [40] Yutian Pang, Sheng Cheng, Jueming Hu, and Yongming Liu. Evaluating the robustness of bayesian neural networks against different types of attacks, 2021.
- [41] V. Passricha and R. K. Aggarwal. A comparative analysis of pooling strategies for convolutional neural network based hindi asr. *Journal of Ambient Intelligence and Humanized Computing*, 11:675–691, 2020.
- [42] Edward Petersen. *Single Event Effects in Aerospace*. Wiley, 2011.
- [43] Carsten Peterson and James R Anderson. A mean field theory learning algorithm for neural networks. *Complex Systems*, 1:995–1019, 1987.
- [44] Varadarajan Rengaraj, Sebastian Jost, Franz Bethke, Christian Plessl, Hossein Mirhosseini, Andrea Walther, and Thomas D. Kühne. A two-step machine learning method for predicting the formation energy of ternary compounds. *Computation*, 11(5), 2023.
- [45] Anqi Xu Rey Wiyatno. Maximal jacobian-based saliency map attack. In *cs.LG*, 2018.
- [46] John A. Rice. *Mathematical statistics and data analysis / John A. Rice*. Thomson/Brooks Cole, Belmont, Calif, 3rd ed., international ed. edition, 2007.
- [47] Fernando Fernandes dos Santos, Pedro Foletto Pimenta, Caio Lunardi, Lucas Draghetti, Luigi Carro, David Kaeli, and Paolo Rech. Analyzing and increasing the reliability of convolutional neural networks on gpus. *IEEE Transactions on Reliability*, 68(2):663–677, 2019.
- [48] Muhammad Shafique, Mahum Naseer, Theocharis Theocharides, Christos Kyrkou, Onur Mutlu, Lois Orosa, and Jungwook Choi. Robust machine learning systems: Challenges, current trends, perspectives, and the road ahead. *CoRR*, abs/2101.02559, 2021.
- [49] Dong Su, Huan Zhang, Hongge Chen, Jinfeng Yi, Pin-Yu Chen, and Yupeng Gao. Is robustness the cost of accuracy? - A comprehensive study on the robustness of 18 deep image classification models. *CoRR*, abs/1808.01688, 2018.
- [50] Max Welling and Yee Whye Teh. Bayesian learning via stochastic gradient langevin dynamics. In *Proceedings of the 28th International Conference on International Conference on Machine Learning, ICML’11*, page 681–688, Madison, WI, USA, 2011. Omnipress.
- [51] Zheyu Yan, Yiyu Shi, Wang Liao, Masanori Hashimoto, Xichuan Zhou, and Cheng Zhuo. When single event upset meets deep neural networks: Observations, explorations, and remedies. In *2020 25th Asia and South Pacific Design Automation Conference (ASP-DAC)*, pages 163–168, 2020.

- [52] Afia Zafar, Muhammad Aamir, Nazri Mohd Nawi, Ali Arshad, Saman Riaz, Abdulrahman Alruban, Ashit Kumar Dutta, and Sultan Almotairi. A comparison of pooling methods for convolutional neural networks. *Applied Sciences*, 12(17), 2022.
- [53] Jiaru Zhang, Yang Hua, Zhengui Xue, Tao Song, Chengyu Zheng, Ruhui Ma, and Haibing Guan. Robust bayesian neural networks by spectral expectation bound regularization. In *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 3814–3823, 2021.

Appendix

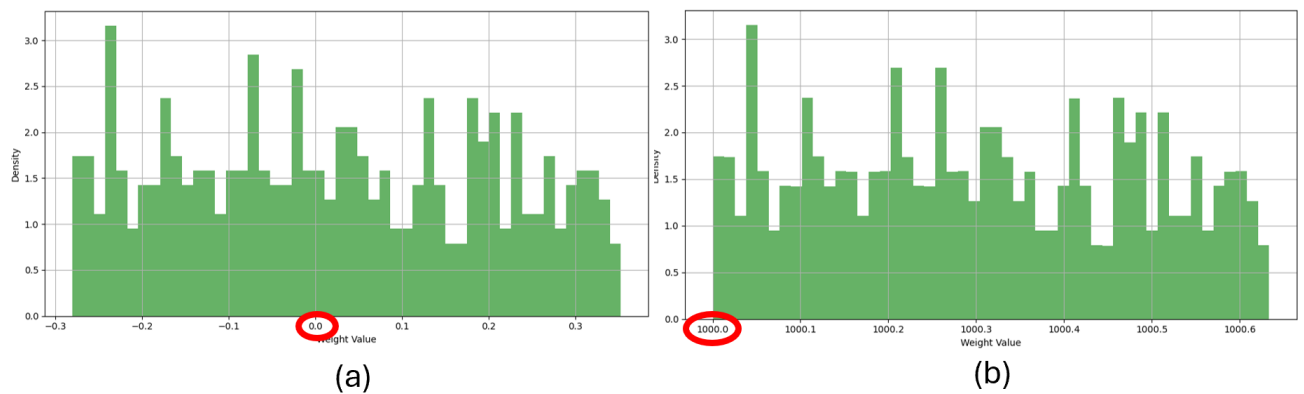
Floating Point Numbers Architecture

Figure A.1: Architecture for storing floating point numbers in this research, which is FP32



Sampled Weight Distribution Illustration

Figure A.2: Sampled Weight Distribution before SEU (panel a), Uniform(-0.28,0.64) and after SEU (panel b), Uniform(1000.0,0.64)



Train Accuracy of Bayesian CNN models

Model Variant	prior	RWG	WG	relu	relu6	sigmoid	sin	tanh	average
Model 0	Gaussian	.7469	.7100	.7503	.7503	.7434	.7172	.7150	.7333
Model 0	Laplace	.6559	.6056	.7172	.7212	.7178	.6266	.6141	.6655
Model 0	Uniform	.9216	.9350	.9306	.9300	.8625	.9316	.9131	.9178
Model 1	Gaussian	.7469	.7100	.7506	.7506	.7434	.7172	.7144	.7333
Model 1	Laplace	.6562	.6044	.7169	.7241	.7184	.6234	.6138	.6653
Model 1	Uniform	.9344	.9297	.9291	.9275	.8625	.9313	.9131	.9182
Model 2	Gaussian	.7434	.6872	.7441	.7438	.7359	.6875	.6887	.7187
Model 2	Laplace	.6697	.6128	.7194	.7225	.7169	.6453	.6197	.6723
Model 2	Uniform	.9319	.9434	.9181	.9297	.8581	.9281	.9275	.9196
Model 3	Gaussian	.7469	.7106	.7503	.7506	.7434	.7172	.7144	.7333
Model 3	Laplace	.6569	.6034	.7156	.7216	.7188	.6212	.6147	.6646
Model 3	Uniform	.9256	.9331	.9300	.9278	.8625	.9319	.9228	.9191
Average		.7780	.7488	.7977	.8000	.7736	.7565	.7476	.7717

Table A.1: Train accuracy of different activation functions across model variants in evaluating Bayesian CNN Models b=0.1

Model Variant	prior	RWG	WG	relu	relu6	sigmoid	sin	tanh	average
Model 0	Gaussian	.6909	.7678	.8716	.8250	.6994	.8231	.7894	.7810
Model 0	Laplace	.6678	.6991	.8703	.7831	.6975	.7741	.7706	.7518
Model 0	Uniform	.7459	.7778	.8684	.8363	.7975	.8694	.8116	.8153
Model 1	Gaussian	.6984	.7684	.8662	.8125	.6966	.8191	.7772	.7769
Model 1	Laplace	.6781	.6953	.8469	.7863	.6963	.7750	.7656	.7491
Model 1	Uniform	.7434	.7800	.8744	.8153	.7947	.8756	.8194	.8147
Model 2	Gaussian	.7206	.7803	.8884	.8631	.6944	.8559	.8306	.8048
Model 2	Laplace	.7013	.7312	.8788	.8500	.6866	.7959	.7978	.7774
Model 2	Uniform	.7366	.7816	.8862	.8694	.8025	.8903	.8072	.8248
Model 3	Gaussian	.6947	.7769	.8719	.8237	.6950	.8184	.7772	.7797
Model 3	Laplace	.6809	.7006	.8541	.7816	.6922	.7709	.7653	.7494
Model 3	Uniform	.6737	.7688	.8747	.8128	.7919	.8759	.8100	.8011
Average		.7027	.7523	.8710	.8216	.7287	.8286	.7935	.7855

Table A.2: Train accuracy of different activation functions across model variants in evaluating Bayesian CNN Models b=1.0

Model Variant	prior	RWG	WG	relu	relu6	sigmoid	sin	tanh	average
Model 0	Gaussian	.6453	.6488	.8328	.6822	.6944	.6691	.7797	.7075
Model 0	Laplace	.6362	.6484	.8228	.6822	.6744	.6647	.7672	.6994
Model 0	Uniform	.6784	.6969	.8666	.6706	.8169	.6966	.8178	.7491
Model 1	Gaussian	.6512	.6538	.7175	.7025	.7000	.6744	.7806	.6971
Model 1	Laplace	.6481	.6494	.7056	.6922	.6731	.6669	.7659	.6859
Model 1	Uniform	.6803	.7078	.8259	.6741	.8222	.7034	.8200	.7477
Model 2	Gaussian	.6872	.6919	.8331	.7356	.7228	.6969	.8122	.7400
Model 2	Laplace	.6750	.6700	.8134	.7156	.7094	.6634	.8016	.7212
Model 2	Uniform	.7094	.7212	.8869	.7794	.8541	.6878	.8425	.7830
Model 3	Gaussian	.6575	.6556	.8347	.6959	.7006	.6847	.7809	.7157
Model 3	Laplace	.6353	.6581	.8187	.6847	.6719	.6847	.7681	.7031
Model 3	Uniform	.6728	.6956	.8656	.6831	.8284	.6956	.8191	.7515
Average		.6647	.6748	.8186	.6998	.7390	.6823	.7963	.7251

Table A.3: Train accuracy of different activation functions across model variants in evaluating Bayesian CNN Models b=10.0

Test Accuracy of Bayesian CNN models

Model Variant	prior	RWG	WG	relu	relu6	sigmoid	sin	tanh	average
Model 0	Gaussian	.7500	.7225	.7837	.7850	.7500	.7588	.7475	.7568
Model 0	Laplace	.7500	.7075	.7700	.7688	.7500	.7438	.7150	.7436
Model 0	Uniform	.9187	.9363	.9213	.9250	.8413	.9387	.8988	.9114
Model 1	Gaussian	.7500	.7225	.7850	.7850	.7500	.7588	.7475	.7570
Model 1	Laplace	.7500	.7087	.7700	.7700	.7500	.7475	.7150	.7445
Model 1	Uniform	.9100	.9313	.9237	.9187	.8387	.9387	.8988	.9086
Model 2	Gaussian	.7500	.7412	.7825	.7788	.7500	.7638	.7512	.7596
Model 2	Laplace	.7462	.7175	.7987	.7950	.7400	.7512	.7312	.7543
Model 2	Uniform	.9213	.9500	.9113	.9113	.8363	.7975	.9137	.8916
Model 3	Gaussian	.7500	.7225	.7850	.7863	.7500	.7588	.7488	.7573
Model 3	Laplace	.7500	.7050	.7700	.7688	.7500	.7425	.7175	.7434
Model 3	Uniform	.9237	.8662	.9225	.9062	.8387	.9413	.9062	.9007
Average		.8058	.7859	.8270	.8249	.7787	.8034	.7909	.8024

Table A.4: Test accuracy of different activation functions across model variants in evaluating Bayesian CNN Models b=0.1

Model Variant	prior	RWG	WG	relu	relu6	sigmoid	sin	tanh	average
Model 0	Gaussian	.7562	.7650	.9137	.8825	.8025	.7612	.8938	.8250
Model 0	Laplace	.7575	.7538	.8975	.8562	.7675	.7500	.8750	.8082
Model 0	Uniform	.8063	.8263	.9025	.8712	.8237	.8762	.8662	.8532
Model 1	Gaussian	.7600	.7612	.9137	.8788	.8050	.7512	.8925	.8232
Model 1	Laplace	.7475	.7550	.9113	.8700	.7650	.7500	.8762	.8107
Model 1	Uniform	.8325	.8125	.9125	.8762	.8237	.9150	.8875	.8657
Model 2	Gaussian	.7525	.7588	.9100	.8925	.7525	.7825	.8862	.8193
Model 2	Laplace	.7488	.7588	.9100	.8825	.7488	.7512	.8738	.8105
Model 2	Uniform	.7400	.8213	.9150	.8875	.8225	.9175	.8662	.8529
Model 3	Gaussian	.7538	.7612	.9200	.8825	.7963	.7625	.8925	.8241
Model 3	Laplace	.7500	.7488	.9175	.8738	.7625	.7512	.8812	.8121
Model 3	Uniform	.7362	.8225	.9237	.7512	.8000	.9163	.9075	.8368
Average		.7618	.7787	.9123	.8671	.7892	.8071	.8832	.8285

Table A.5: Test accuracy of different activation functions across model variants in evaluating Bayesian CNN Models b=1

Model Variant	prior	RWG	WG	relu	relu6	sigmoid	sin	tanh	average
Model 0	Gaussian	.7262	.7300	.8550	.8237	.8100	.7425	.8738	.7945
Model 0	Laplace	.7562	.7550	.8612	.8113	.7712	.7488	.8700	.7962
Model 0	Uniform	.7612	.7975	.9075	.7600	.8962	.7500	.9062	.8255
Model 1	Gaussian	.7538	.7488	.7913	.8137	.8275	.7475	.8738	.7938
Model 1	Laplace	.7488	.7462	.7850	.7950	.7662	.7500	.8688	.7800
Model 1	Uniform	.7488	.8287	.9075	.8137	.8912	.7500	.9150	.8364
Model 2	Gaussian	.7488	.7500	.8562	.8087	.7675	.7500	.8850	.7952
Model 2	Laplace	.7488	.7512	.8488	.7825	.7550	.7488	.8600	.7850
Model 2	Uniform	.7775	.7900	.8988	.8688	.8912	.7500	.9062	.8404
Model 3	Gaussian	.7525	.7500	.8575	.8213	.8187	.7488	.8812	.8043
Model 3	Laplace	.7500	.7562	.8612	.8050	.7700	.7500	.8688	.7945
Model 3	Uniform	.7750	.8050	.9113	.8037	.8962	.7500	.9125	.8362
Average		.7540	.7674	.8618	.8090	.8218	.7489	.8851	.8068

Table A.6: Test accuracy of different activation functions across model variants in evaluating Bayesian CNN Models b=10

Robustness of Deterministic and Bayesian CNN models

Figure A.3: Robustness of Deterministic CNN model by model variant

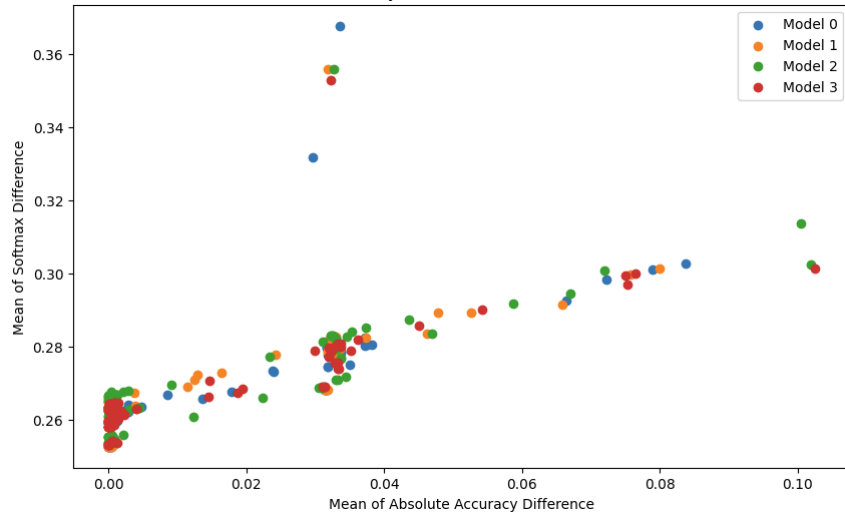
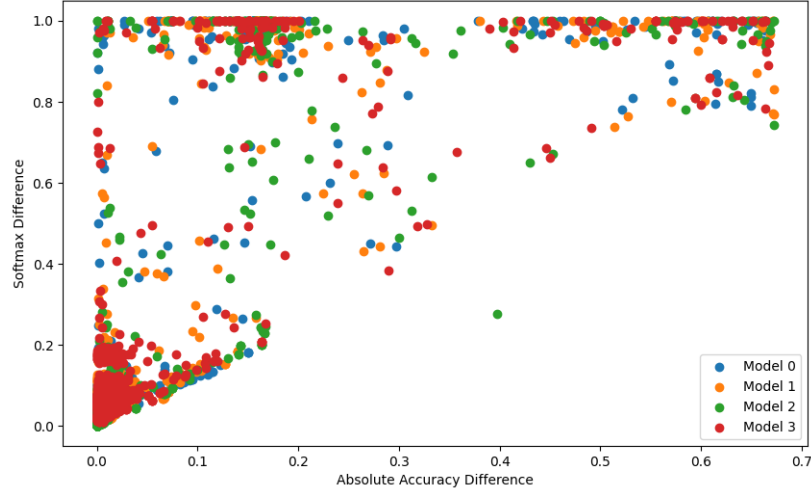


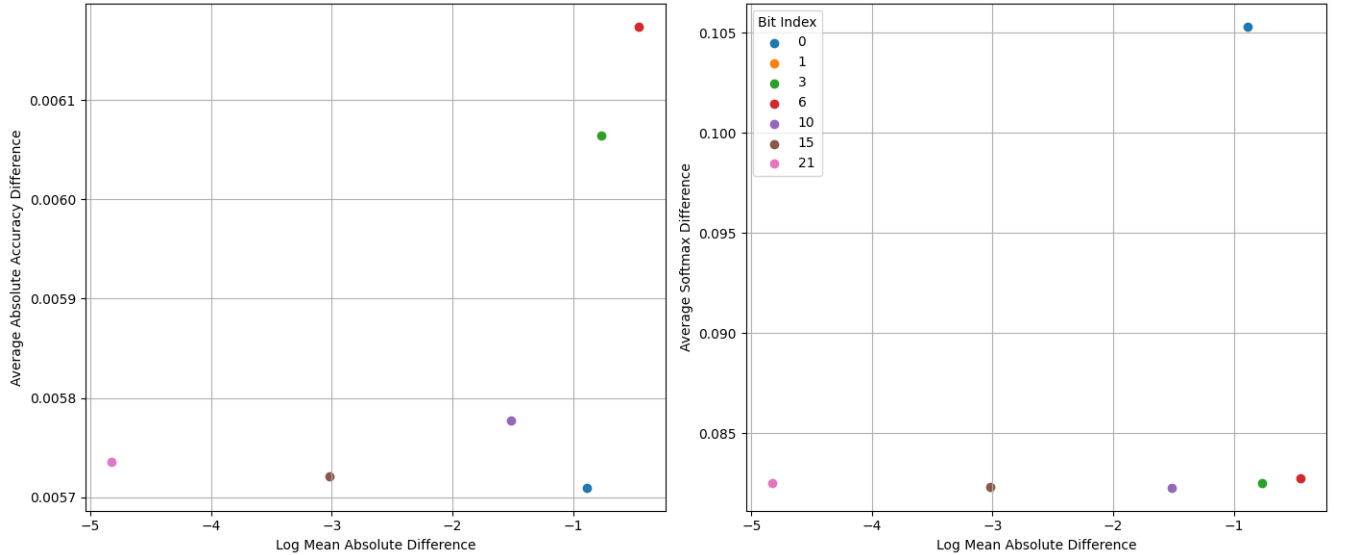
Figure A.4: Robustness of Bayesian CNN model by model variant



Bit Index	After Value
0	0.60335773229599
1	-2.0531199724572943e+38
3	-1.404801691640145e-10
6	-0.037709858268499374
10	-0.72835773229599
15	-0.60726398229599
21	-0.60329669713974

Table A.7: Illustration of value change after SEU (original value = -0.60335773229599)

Figure A.5: Log mean absolute difference against robustness metrics, excluding the data from bit index=1



GitHub Repository

The source code used in this research is available at <https://github.com/putawararevalda/bnndesignrobustness>.